

Zbiór zadań - Python

Piotr Jastrzębski

2026-10-13

Spis treści

Zbiór zadań - Python 4

I Wprowadzenie do języka Python 5

1 Zmienne, typy i operacje wejścia/wyjścia 6

2 Instrukcja warunkowa 9

3 Instrukcja iteracyjna 11

4 Listy 14

5 Tablice 18

6 Krotki 21

7 Zbiory 25

8 Słowniki 28

9 Funkcje 30

10 Argumenty funkcji 33

11 f-stringi 36

12 Obsługa wyjątków 38

13 Obsługa plików 41
13.1 Pliki tekstowe 41
13.2 Pliki binarne 43

14 Struktura programu i moduły 44

15 Zasady pisania czystego kodu 47

II Programowanie obiektowe 51

16 Proste klasy 52

17 Inicjalizator 54

18 Właściwości 57

19 Dziedziczenie i polimorfizm 60

20 Reprezentacje tekstowe obiektów 63

21 Emulacja kontenerów 65

22 Emulacja arytmetyki 67

23 Emulacja porównania 70

III Zaawansowane zagadnienia 72

24 Dekoratory 73

25 Wyrażenia regularne 76

26 Iteratory i generatory 78

27 Operatory bitowe 80

28 Wybrane typy sekwencyjne 83

29 Wyrażenia lambda 86

30 Moduł `functools` 88

Zbiór zadań - Python

Tu będzie zbiór zadań z programowania w języku Python. Inspiracją było zebranie zadań powstałych w trakcie prowadzenia zajęć dydaktycznych realizowanych na Wydziale Matematyki i Informatyki Uniwersytetu Warmińsko-Mazurskiego w Olsztynie.

Niektóre treści zadań generowane za pomocą ChatGPT, Claude, Gemini lub DeepSeek.

Część I

Wprowadzenie do języka Python

1 Zmienne, typy i operacje wejścia/wyjścia

W zadaniach z tego rozdziału korzystaj wyłącznie z sekwencji instrukcji — bez instrukcji warunkowych i pętli.

1. Napisz program, który wypisze na ekranie trzy linie: Twoje imię, nazwę kierunku studiów oraz nazwę uczelni. Każdą linię poprzedź komentarzem (`#`) opisującym, co wypisuje.
2. Za pomocą jednego wywołania funkcji `print()` wypisz słowa `"Python"`, `"jest"`, `"super"` tak, aby były oddzielone myślnikiem (`-`), a na końcu linii pojawił się wykrzyknik zamiast znaku nowej linii. Wykorzystaj parametry `sep` i `end`.
3. Utwórz zmienne `first_name`, `age`, `height` i `is_student`, przypisując im kolejno: napis, liczbę całkowitą, liczbę zmiennoprzecinkową oraz wartość logiczną. Wypisz każdą zmienną wraz z jej typem, korzystając z funkcji `type()`.
4. Wczytaj od użytkownika imię za pomocą `input()` i wypisz powitanie w postaci `Cześć, <imię>!`.
5. Wczytaj od użytkownika dwie liczby całkowite (pamiętaj o konwersji `int()`), a następnie wypisz ich sumę, różnicę, iloczyn, iloraz (`/`), iloraz całkowity (`//`), resztę z dzielenia (`%`) oraz wynik potęgowania (`**`).
6. Sprawdź i wypisz wyniki wyrażeń:

```
7 / 2
7 // 2
-7 // 2
7 % 3
-7 % 3
2 ** 10
0.1 + 0.2
```

W komentarzach wyjaśnij wyniki, które Cię zaskoczyły.

7. Wczytaj od użytkownika długość boku kwadratu (liczba zmiennoprzecinkowa) i wypisz jego pole oraz obwód. Wynik zaokrąglij do dwóch miejsc po przecinku za pomocą funkcji `round()`.

8. Wczytaj od użytkownika temperaturę w stopniach Celsjusza i przelicz ją na stopnie Fahrenheita według wzoru $F = \frac{9}{5}C + 32$. Wypisz wynik.

9. Wczytaj od użytkownika liczbę sekund (liczba całkowita) i przelicz ją na godziny, minuty i sekundy, korzystając z operatorów `//` i `%`. Przykładowo dla `3725` program powinien wypisać `1 h 2 min 5 s`.

10. Wczytaj od użytkownika liczbę trzycyfrową i wypisz osobno cyfrę setek, dziesiątek i jedności, korzystając wyłącznie z operatorów `//` i `%`. Następnie wypisz sumę tych cyfr.

11. Utwórz zmienne `a = 5` i `b = 3`. Wykonaj kolejno operacje przypisania złożonego: `a += b`, `a *= 2`, `a -= 1`, `a //= 3`, `a **= 2`, wypisując wartość `a` po każdej z nich.

12. Sprawdź, co się stanie, gdy spróbujesz dodać napis `"10"` do liczby `5`. Odczytaj komunikat błędu i w komentarzu wyjaśnij jego przyczynę. Następnie popraw kod na dwa sposoby: tak, aby wynikiem było `15`, oraz tak, aby wynikiem było `"105"`.

13. Wypisz wyniki konwersji: `int("42")`, `float("3.14")`, `int(3.99)`, `int(-3.99)`, `str(2026)`, `bool(0)`, `bool(15)`, `bool("")`, `bool("False")`. W komentarzach opisz, które wartości są traktowane jako fałsz logiczny.

14. Dany jest napis:

```
text = "  Wstęp do Programowania  "
```

Wypisz: długość napisu (`len()`), napis bez białych znaków na początku i końcu (`.strip()`), napis zapisany wielkimi literami (`.upper()`), napis zapisany małymi literami (`.lower()`) oraz napis, w którym słowo `"Programowania"` zastąpiono słowem `"Pythona"` (`.replace()`).

15. Dany jest napis `word = "informatyka"`. Wypisz jego pierwszy znak, ostatni znak (z użyciem indeksu ujemnego) oraz liczbę wystąpień litery `"a"` (`.count()`). Sprawdź też, co zwraca `word.find("ma")`.

16. Wczytaj od użytkownika imię i nazwisko (dwa osobne wywołania `input()`) i wypisz inicjały w postaci `J.K.`, korzystając z indeksowania napisów i operatora `+`.

17. Za pomocą operatora `*` dla napisów wypisz linię złożoną z 40 znaków `=`, pod nią tytuł `RAPORT`, a pod nim ponownie linię z 40 znaków `=`.

18. Wczytaj od użytkownika kwotę w złotych (liczba zmiennoprzecinkowa) oraz kurs euro. Wypisz kwotę po przeliczeniu na euro, zaokrągloną do dwóch miejsc po przecinku.

19. Wczytaj od użytkownika współrzędne dwóch punktów na płaszczyźnie: `x1`, `y1`, `x2`, `y2`. Oblicz i wypisz odległość między punktami, korzystając z operatora `** 0.5` zamiast pierwiastka.

20. Napisz program obliczający wskaźnik BMI. Wczytaj masę ciała w kilogramach i wzrost w centymetrach, przelicz wzrost na metry, oblicz $BMI = \frac{masa}{wzrost^2}$ i wypisz wynik zaokrąglony do jednego miejsca po przecinku. Zadbaj o czytelne komunikaty przy wczytywaniu danych.

2 Instrukcja warunkowa

1. Wczytaj od użytkownika liczbę całkowitą i wypisz, czy jest dodatnia, ujemna, czy równa zero.
2. Wczytaj od użytkownika liczbę całkowitą i wypisz, czy jest parzysta, czy nieparzysta (wykorzystaj operator `%`).
3. Wczytaj od użytkownika wiek i wypisz komunikat `"Pełnoletni"` lub `"Niepełnoletni"`. Następnie zapisz to samo rozwiązanie w jednej linii, korzystając z wyrażenia warunkowego (`x if warunek else y`).
4. Wczytaj od użytkownika dwie liczby i wypisz większą z nich bez użycia funkcji `max()`. Jeśli liczby są równe, wypisz odpowiedni komunikat.
5. Wczytaj od użytkownika trzy liczby i wypisz największą z nich, korzystając wyłącznie z instrukcji `if`, `elif` i `else` oraz operatorów porównania.
6. Wczytaj od użytkownika liczbę punktów (od 0 do 100) uzyskanych na kolokwium i wypisz ocenę według skali:

   - 0–50 — `2.0`,
   - 51–60 — `3.0`,
   - 61–70 — `3.5`,
   - 71–80 — `4.0`,
   - 81–90 — `4.5`,
   - 91–100 — `5.0`.

   Jeśli liczba punktów jest spoza zakresu, wypisz komunikat o błędnych danych.
7. Wczytaj od użytkownika rok i sprawdź, czy jest przestępny. Rok jest przestępny, jeśli jest podzielny przez 4 i niepodzielny przez 100 lub jest podzielny przez 400. Zapisz warunek w jednym wyrażeniu z użyciem operatorów `and`, `or`.
8. Wczytaj od użytkownika temperaturę i za pomocą porównania łańcuchowego (np. `15 <= t < 25`) wypisz, czy jest `"zimno"` (poniżej 15), `"przyjemnie"` (od 15 do 25) czy `"gorąco"` (powyżej 25).
9. Wczytaj od użytkownika pojedynczy znak i sprawdź, czy jest samogłoską (`a`, `e`, `i`, `o`, `u`, `y`), korzystając z operatora `in` dla napisu `"aeiouy"`. Uwzględnij również wielkie litery (np. przez `.lower()`).

10. Wczytaj od użytkownika długości trzech odcinków i sprawdź, czy można z nich zbudować trójkąt (suma długości dwóch dowolnych boków musi być większa od trzeciego). Jeśli tak, określ, czy trójkąt jest równoboczny, równoramienny czy różnoboczny.

11. Wczytaj od użytkownika współczynniki `a`, `b`, `c` równania kwadratowego $ax^2 + bx + c = 0$. Oblicz wyróżnik Δ i wypisz pierwiastki równania (dwa, jeden lub informację o ich braku w zbiorze liczb rzeczywistych). Obsłuż osobno przypadek `a == 0`.

12. Wczytaj od użytkownika współrzędne punktu `x`, `y` i wypisz, w której ćwiartce układu współrzędnych leży punkt. Uwzględnij przypadki, gdy punkt leży na osi lub w początku układu.

13. Wczytaj od użytkownika login i hasło. Jeśli login to `"admin"` i hasło to `"python123"`, wypisz `"Zalogowano"`. W przeciwnym razie wypisz, czy błędny był login, czy hasło. Wykorzystaj zagnieżdżoną instrukcję `if`.

14. Wczytaj od użytkownika napis i wypisz informację, czy jest pusty. Wykorzystaj to, że pusty napis ma wartość logiczną `False` (warunek `if not text:`).

15. Wczytaj od użytkownika numer dnia tygodnia (1–7) i wypisz jego nazwę, korzystając z instrukcji `match`–`case`. Dla wartości spoza zakresu użyj wzorca `case _`.

16. Napisz prosty kalkulator: wczytaj dwie liczby oraz symbol operacji (`+`, `-`, `*`, `/`) i wypisz wynik. Do wyboru operacji użyj instrukcji `match`–`case`. Obsłuż przypadek dzielenia przez zero oraz nieznanego symbolu operacji komunikatem dla użytkownika.

17. Wczytaj od użytkownika kwotę zakupów. Sklep udziela rabatu: 5% od kwoty powyżej 100 zł, 10% od kwoty powyżej 500 zł oraz 15% od kwoty powyżej 1000 zł. Dodatkowo, jeśli klient posiada kartę lojalnościową (wczytaj odpowiedź `t`/`n`), otrzymuje dodatkowe 2% rabatu. Wypisz kwotę do zapłaty.

18. Wczytaj od użytkownika masę przesyłki w kilogramach oraz informację, czy jest to przesyłka priorytetowa. Oblicz koszt wysyłki według reguł:

    - do 1 kg — 10 zł,
    - powyżej 1 kg do 5 kg — 15 zł,
    - powyżej 5 kg do 20 kg — 25 zł,
    - powyżej 20 kg przesyłka nie jest przyjmowana.

    Przesyłka priorytetowa kosztuje o 50% więcej.

19. Wczytaj od użytkownika datę w postaci trzech liczb: dzień, miesiąc, rok. Sprawdź, czy data jest poprawna (uwzględnij liczbę dni w miesiącach oraz lata przestępne).

20. Wczytaj od użytkownika godzinę w formacie 24-godzinnym (`godzina` i `minuta` jako dwie liczby całkowite) i wypisz ją w formacie 12-godzinnym z dopiskiem `AM` lub `PM` (np. `0:15` → `12:15 AM`, `13:05` → `1:05 PM`, `12:00` → `12:00 PM`). Sprawdź poprawność danych wejściowych.

3 Instrukcja iteracyjna

W zadaniach z tego rozdziału nie korzystaj z list — wystarczą zmienne, napisy i funkcja `range()`.

1. Za pomocą pętli `for` i funkcji `range()` wypisz liczby od 1 do 10, każdą w osobnej linii. Następnie wypisz je w jednej linii, oddzielone spacją (parametr `end` funkcji `print()`).
2. Za pomocą pętli `for` wypisz:

   - wszystkie liczby parzyste od 0 do 30,
   - liczby od 20 do 0 w kolejności malejącej,
   - co piątą liczbę z zakresu od 5 do 100.

   W każdym przypadku użyj odpowiednich argumentów funkcji `range()` zamiast instrukcji warunkowej.
3. Wczytaj od użytkownika liczbę naturalną `n` i oblicz sumę liczb od 1 do `n` za pomocą pętli. Porównaj wynik ze wzorem $\frac{n(n+1)}{2}$.
4. Wczytaj od użytkownika liczbę naturalną `n` i oblicz jej silnię $n!$ za pomocą pętli `for`.
5. Wczytaj od użytkownika liczbę całkowitą i wypisz dla niej tabliczkę mnożenia od 1 do 10 w postaci `7 x 3 = 21`.
6. Za pomocą pętli zagnieżdżonych wypisz tabliczkę mnożenia 10×10 w postaci tabeli. Do wyrównania kolumn możesz użyć metody `.rjust()` napisów lub znaku tabulacji `"\t"`.
7. Wczytaj od użytkownika napis i za pomocą pętli `for` przechodzącej po znakach napisu policz, ile zawiera samogłosek, a ile cyfr.
8. Wczytaj od użytkownika napis i zbuduj (w pętli, znak po znaku) nowy napis zapisany w odwrotnej kolejności. Nie korzystaj z wycinków.
9. Wczytaj od użytkownika liczbę naturalną i za pomocą pętli `while` oraz operatorów `//` i `%` oblicz sumę jej cyfr oraz liczbę cyfr.
10. Napisz program, który wczytuje od użytkownika liczby tak długo, aż zostanie podane `0`. Na koniec wypisz sumę, liczbę wczytanych wartości (bez zera) oraz ich średnią arytmetyczną. Obsłuż przypadek, gdy pierwszą podaną liczbą jest `0`.

11. Wczytuj od użytkownika liczbę z zakresu od 1 do 10 tak długo, aż poda poprawną wartość (walidacja danych wejściowych w pętli `while`). Po każdej błędnej próbie wypisz komunikat.

12. Wypisz wszystkie liczby od 1 do 100, ale zamiast liczb podzielnych przez 3 wypisz `Fizz`, zamiast podzielnych przez 5 — `Buzz`, a zamiast podzielnych przez 3 i 5 — `FizzBuzz`.

13. Wczytaj od użytkownika liczbę naturalną `n` i sprawdź, czy jest liczbą pierwszą. Przerwij pętlę instrukcją `break` po znalezieniu pierwszego dzielnika. Wykorzystaj klauzulę `else` pętli `for` do wypisania informacji, że liczba jest pierwsza.

14. Wypisz wszystkie liczby pierwsze mniejsze od 100, korzystając z pętli zagnieżdżonych.

15. Za pomocą pętli `for` i instrukcji `continue` wypisz liczby od 1 do 50 z pominięciem liczb podzielnych przez 7 oraz liczb zawierających cyfrę 7 (sprawdź `"7" in str(i)`).

16. Wczytaj od użytkownika liczbę naturalną `n` i wypisz `n` pierwszych wyrazów ciągu Fibonacciego, korzystając wyłącznie z dwóch zmiennych pomocniczych i wielokrotnego przypisania (`a, b = b, a + b`).

17. Wczytaj od użytkownika dwie liczby naturalne i oblicz ich największy wspólny dzielnik algorytmem Euklidesa (w wersji z resztą z dzielenia), korzystając z pętli `while`. Na tej podstawie oblicz też najmniejszą wspólną wielokrotność.

18. Wczytaj od użytkownika liczbę naturalną `n` i wypisz (za pomocą pętli i operatora `*` dla napisów) następujące figury o wysokości `n`:

    - trójkąt prostokątny z gwiazdek wyrównany do lewej,
    - ten sam trójkąt wyrównany do prawej,
    - choinkę (trójkąt równoramienny).

    Przykład choinki dla `n = 3`:

    ```
      *
     ***
    *****
    ```

19. Wczytaj od użytkownika liczbę naturalną i wypisz kolejne wyrazy ciągu Collatza, aż ciąg osiągnie wartość 1 (jeśli liczba jest parzysta, dzielimy ją przez 2, w przeciwnym razie mnożymy przez 3 i dodajemy 1). Na koniec wypisz liczbę wykonanych kroków oraz największą wartość, jaka pojawiła się w ciągu.

20. Napisz grę w zgadywanie liczby. Ustal w kodzie tajną liczbę z zakresu 1–100 (np. `secret = 37`). Użytkownik ma maksymalnie 7 prób; po każdej próbie program wypisuje `"za mało"` lub `"za dużo"`. Po odgadnięciu wypisz liczbę wykorzystanych prób i zakończ pętlę instrukcją `break`. Jeśli wszystkie próby się wyczerpią, wypisz tajną liczbę (wykorzystaj klauzulę `else` pętli).

21. Napisz program z menu tekstowym działający w pętli `while True`. Program przechowuje stan konta (początkowo 0 zł) i pozwala wybrać opcję: `1` — wpłata, `2` — wypłata, `3` — wyświetlenie salda, `0` — wyjście. Nie pozwól na wypłatę większą niż saldo. Pętlę zakończ instrukcją `break`.

4 Listy

1. Utwórz listę `fruits` zawierającą pięć nazw owoców (jako stringi). Wypisz całą listę, a następnie wypisz osobno drugi i ostatni element.
2. Utwórz pustą listę `shopping_list`. Dodaj do niej cztery produkty za pomocą metody `.append()`, a następnie wstaw piąty produkt na początek listy za pomocą `.insert()`. Wypisz wynik.
3. Dana jest lista:

```
colors = ["red", "green", "blue", "yellow", "green", "purple"]
```

Usuń pierwsze wystąpienie `"green"` za pomocą `.remove()`, a następnie usuń element o indeksie `3` za pomocą `del`. Wypisz listę po każdej operacji.

4. Utwórz listę `scores` zawierającą 8 dowolnych liczb całkowitych. Wypisz liczbę elementów (`len()`), ich sumę (`sum()`) oraz średnią arytmetyczną (obliczoną ręcznie na podstawie `sum` i `len`).
5. Dana jest lista:

```
letters = ["a", "b", "c", "d", "e", "f", "g", "h"]
```

Korzystając wyłącznie z wycinków (slicing): - pobierz pierwsze trzy elementy, - pobierz co drugi element, - pobierz elementy od indeksu `2` do `5` (włącznie), - odwróć listę.

6. Dana jest lista:

```
temperatures = [23.1, 18.4, 30.0, 15.7, 22.2, 28.9]
```

Wypisz listę posortowaną rosnąco (bez modyfikacji oryginału — użyj `sorted()`), a następnie posortuj oryginał malejąco w miejscu (`.sort()`). Wypisz obie wersje.

7. Utwórz listę `squares` zawierającą kwadraty liczb od 1 do 15 włącznie, korzystając z list comprehension. Następnie utwórz drugą listę `even_squares`, która zawiera tylko te kwadraty, które są liczbami parzystymi.

8. Dana jest lista:

```
words = ["apple", "hi", "banana", "go", "strawberry", "no", "watermelon"]
```

Przy użyciu list comprehension utwórz nową listę `long_words`, która zawiera tylko słowa dłuższe niż 4 znaki.

9. Utwórz listę dwuwymiarową `matrix` o wymiarach 3×3, wypełnioną liczbami od 1 do 9. Wypisz:

- cały wiersz o indeksie `1`,
- element znajdujący się w wierszu `2`, kolumnie `0`,
- wszystkie elementy z przekątnej głównej (indeksy `[0][0]`, `[1][1]`, `[2][2]`).

10. Dane są dwie listy:

```
student_names = ["Alice", "Bob", "Charlie", "Diana"]
student_grades = [85, 92, 78, 95]
```

Połącz je w listę krotek `student_records` za pomocą `zip()`. Następnie wypisz imiona studentów, którzy uzyskali ocenę wyższą niż 80.

11. Dana jest lista:

```
cities = ["Warsaw", "Berlin", "Prague", "Vienna", "Budapest", "Prague", "Rome"]
```

Używając `enumerate()`, wypisz wszystkie indeksy, pod którymi występuje wartość `"Prague"`.

12. Utwórz listę zagnieżdżoną:

```
original = [[1, 2], [3, 4], [5, 6]]
```

Wykonaj kopię płytką (`shallow_copy`) i kopię głęboką (`deep_copy`). Zmodyfikuj element `original[0][0] = 99`. Wypisz wszystkie trzy listy i wyjaśnij (komentarzem w kodzie), dlaczego wyniki się różnią.

13. Dana jest lista:

```
nested = [[1, 2, 3], [4, 5], [6, 7, 8, 9], [10]]
```

Napisz dwa rozwiązania, które spłaszczą ją do jednej listy `flat_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`: 1. za pomocą zagnieżdżonego list comprehension, 2. za pomocą pętli `for`.

14. Dana jest lista liczb:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

Napisz funkcję `chunk_list(data, chunk_size)`, która podzieli listę na mniejsze podlisty o zadanym rozmiarze. Przykład wywołania:

```
chunk_list(numbers, 4)
# oczekiwany wynik: [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

15. Dana jest macierz (lista list):

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

Napisz kod, który utworzy `transposed_matrix` — transpozycję powyższej macierzy — korzystając z `zip()` i list comprehension. Wypisz wynik.

16. Dana jest lista:

```
raw_data = [3, 5, 2, 3, 7, 5, 8, 2, 9, 7, 1, 3]
```

Utwórz listę `unique_data`, która zawiera te same elementy bez duplikatów, zachowując oryginalną kolejność pierwszego wystąpienia. Nie korzystaj z `set()` w sposób, który zmienia kolejność.

17. Dana jest lista stringów:

```
raw_prices = ["12.50", "abc", "7.99", "", "25.00", "not_a_number", "3.14"]
```

1. Za pomocą funkcji pomocniczej i `filter()` odfiltruj elementy, które nie dają się przekonwertować na `float`.
2. Za pomocą `map()` przekonwertuj przefiltrowane elementy na typ `float`.
3. Wynik zapisz w liście `valid_prices`.
4. Korzystając wyłącznie z wbudowanej listy Pythona i jej metod (`.append()`, `.pop()`, `.pop(0)`):
5. Zasymuluj działanie stosu (LIFO) — dodaj 5 elementów, a następnie zdejmij 3.

6. Zasymuluj działanie kolejki (FIFO) — dodaj 5 elementów, a następnie pobierz 3.

Wypisz stan struktury po każdej operacji.

19. Dana jest lista słowników:

```
employees = [
    {"name": "Alice", "department": "IT", "salary": 7200},
    {"name": "Bob", "department": "HR", "salary": 5400},
    {"name": "Charlie", "department": "IT", "salary": 6800},
    {"name": "Diana", "department": "HR", "salary": 5900},
    {"name": "Eve", "department": "IT", "salary": 7200},
]
```

Posortuj listę: 1. Rosnąco według `department`, a w ramach tego samego działu — malejąco według `salary`. 2. Użyj parametru `key` w funkcji `sorted()` z wyrażeniem `lambda`.

20. Napisz dwa warianty kodu, które znajdują pierwszą liczbę podzielną przez 7 i 13 w zakresie od 1 do 10 000 000:

- List comprehension — najpierw generuje pełną listę, potem pobiera pierwszy element.
- Generator expression — korzysta z `next()` na wyrażeniu generatorowym.

Zmierz czas wykonania obu wariantów za pomocą modułu `time` i wypisz wyniki. W komentarzu wyjaśnij, dlaczego różnią się wydajnością.

5 Tablice

W zadaniach z tego rozdziału korzystaj z typu `array` ze standardowego modułu `array` (`from array import array`). W odróżnieniu od listy tablica przechowuje elementy tylko jednego, ustalonego typu liczbowego, określonego kodem typu (np. `'i'` — liczby całkowite, `'d'` — liczby zmiennoprzecinkowe).

1. Utwórz tablicę `numbers` z kodem typu `'i'` zawierającą liczby `[4, 8, 15, 16, 23, 42]`. Wypisz tablicę, jej długość, pierwszy i ostatni element.
2. Utwórz tablicę `measurements` z kodem typu `'d'` zawierającą pięć dowolnych pomiarów. Wypisz jej atrybuty `typecode` oraz `itemsize` (rozmiar jednego elementu w bajtach). Porównaj `itemsize` z tablicą o kodzie typu `'i'`.
3. Utwórz pustą tablicę o kodzie typu `'i'`. Za pomocą pętli `for` i metody `.append()` dodaj do niej kwadraty liczb od 1 do 10. Następnie za pomocą metody `.extend()` dodaj na raz liczby z listy `[200, 300, 400]`. Wypisz wynik.
4. Dana jest tablica:

```
values = array('i', [10, 20, 30, 40, 50, 60, 70])
```

Korzystając z wycinków (slicing), wypisz: pierwsze trzy elementy, co drugi element oraz elementy w odwrotnej kolejności. Sprawdź typ wyniku wycinka.

5. Dla tablicy z poprzedniego zadania:

   - zmień element o indeksie `2` na `35`,
   - wstaw liczbę `5` na początek tablicy (`.insert()`),
   - usuń pierwsze wystąpienie liczby `40` (`.remove()`),
   - usuń i wypisz ostatni element (`.pop()`).

   Wypisz tablicę po każdej operacji.
6. Utwórz tablicę o kodzie typu `'i'` i spróbuj dodać do niej:

   - liczbę zmiennoprzecinkową `3.5`,
   - napis `"7"`.

Odczytaj komunikaty błędów i w komentarzach wyjaśnij, czym tablica różni się pod tym względem od listy. Sprawdź też, co się stanie, gdy do tablicy o kodzie `'d'` dodasz liczbę całkowitą.

7. Utwórz tablicę o kodzie typu `'b'` (liczby całkowite 1-bajtowe ze znakiem) i spróbuj dodać do niej liczby `127`, `128` i `-129`. Opisz w komentarzu, z czego wynika ograniczenie zakresu wartości.

8. Dana jest tablica:

```
grades = array('i', [3, 5, 4, 5, 2, 5, 4, 3, 5])
```

Wypisz, ile razy występuje ocena `5` (`.count()`), indeks pierwszego wystąpienia oceny `2` (`.index()`), sumę i średnią ocen oraz ocenę najwyższą i najniższą (`max()`, `min()`).

9. Dana jest lista `prices = [19.99, 5.49, 3.20, 12.00, 7.75]`. Utwórz na jej podstawie tablicę o kodzie typu `'d'`, następnie w pętli zwiększ każdą cenę o 23% (VAT), modyfikując elementy tablicy przez indeks. Na koniec przekształć tablicę z powrotem na listę metodą `.tolist()` i wypisz wynik.

10. Dana jest tablica:

```
temperatures = array('d', [12.5, 14.1, 9.8, 15.3, 17.0, 11.2, 13.4])
```

Za pomocą pętli `for` znajdź (bez użycia `max()` i `min()`) indeks dnia z najwyższą i najniższą temperaturą. Policz także, w ilu dniach temperatura była wyższa od średniej.

11. Utwórz tablicę o kodzie typu `'i'` zawierającą liczby od 1 do 20. Za pomocą list comprehension utwórz z niej nową tablicę zawierającą tylko liczby podzielne przez 3. Odwróć ją w miejscu metodą `.reverse()` i wypisz wynik.

12. Dane są dwie tablice o tej samej długości:

```
a = array('d', [1.0, 2.0, 3.0])
b = array('d', [4.0, 5.0, 6.0])
```

Oblicz i wypisz:

- tablicę `c` będącą sumą odpowiadających sobie elementów,
- iloczyn skalarny wektorów `a` i `b`.

Wykorzystaj `zip()`. Sprawdź też, co daje wyrażenie `a + b` i dlaczego nie jest to suma wektorowa.

13. Wczytuj od użytkownika liczby całkowite tak długo, aż poda pusty napis, i zapisuj je w tablicy o kodzie typu `'i'`. Następnie posortuj elementy (funkcja `sorted()` zwraca listę — utwórz z niej nową tablicę) i wypisz medianę.

14. Utwórz tablicę `data_array` (kod `'i'`) zawierającą liczby od 0 do 9999 i wypisz, ile bajtów zajmują przechowywane w niej dane (`len(data_array) * data_array.itemsize`). W komentarzu wyjaśnij, dlaczego lista z tymi samymi liczbami zajmuje znacznie więcej pamięci (wskazówka: co tak naprawdę przechowuje lista?).

6 Krotki

1. Utwórz krotkę `weekdays` zawierającą nazwy pięciu dni roboczych (jako stringi). Wypisz całą krotkę, a następnie wypisz osobno pierwszy i ostatni element.
2. Utwórz pustą krotkę `empty_tuple` oraz krotkę jednoelementową `single_item` przechowującą liczbę `42`. Wypisz obie krotki wraz z ich typem (`type()`). Sprawdź, co się stanie, jeśli pominiesz przecinek w krotce jednoelementowej, i wyjaśnij wynik komentarzem.
3. Dana jest krotka:

```
months = ("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")
```

Korzystając wyłącznie z wycinków (slicing): - pobierz miesiące letnie (Jun–Aug), - pobierz co trzeci miesiąc, - pobierz ostatnie cztery miesiące, - odwróć kolejność elementów.

4. Dana jest krotka:

```
coordinates = (52.23, 21.01)
```

Spróbuj zmienić drugi element na `19.94`. Przechwyć wyjątek `TypeError` za pomocą bloku `try/except` i wypisz komunikat wyjaśniający, dlaczego operacja się nie powiodła.

5. Dana jest krotka:

```
grades = (4, 5, 3, 5, 4, 5, 2, 4, 5, 3)
```

Wypisz, ile razy występuje ocena `5` (`.count()`), oraz podaj indeks pierwszego wystąpienia oceny `2` (`.index()`).

6. Dana jest krotka:

```
person = ("Alice", 30, "developer", "Warsaw")
```

Rozpakuj ją do zmiennych `name`, `age`, `role`, `city` w jednej linii. Następnie rozpakuj ponownie, ale tym razem przypisz tylko imię do `name`, a resztę do zmiennej `rest` za pomocą operatora `*`.

7. Utwórz dwie zmienne `x = 10` i `y = 20`. Zamień ich wartości miejscami w jednej linii, korzystając z mechanizmu pakowania/rozpakowywania krotek (bez zmiennej tymczasowej). Wypisz wartości przed zamianą i po niej.

8. Utwórz słownik `city_distances`, w którym kluczami są krotki dwóch miast, a wartościami — odległości w kilometrach, np.:

```
("Warsaw", "Berlin"): 573
```

Dodaj co najmniej 4 wpisy. Napisz kod, który na podstawie podanej pary miast wypisuje odległość lub komunikat, że trasa nie istnieje.

9. Dana jest krotka:

```
products = ("laptop", "mouse", "keyboard", "monitor", "headset")
```

Używając `enumerate()` z parametrem `start=1`, wypisz ponumerowaną listę produktów w formacie:

```
1. laptop
2. mouse
...
```

10. Dane są dwie listy:

```
country_names = ["Poland", "Germany", "France", "Spain"]
country_codes = ["PL", "DE", "FR", "ES"]
```

Połącz je w krotkę krotek `country_data` za pomocą `zip()` opakowanego w `tuple()`. Wypisz wynik oraz oddzielnie trzeci element.

11. Dana jest lista krotek:

```
runners = [("Alice", 12.5), ("Bob", 11.8), ("Charlie", 13.1), ("Diana", 11.2)]
```

Posortuj listę rosnąco według czasu (drugi element krotki), korzystając z `sorted()` i parametru `key` z wyrażeniem `lambda`. Wypisz posortowaną listę.

12. Zaimportuj `namedtuple` z modułu `collections`. Utwórz typ `Book` z polami `title`, `author`, `year`, `pages`. Stwórz trzy instancje książek i umieść je w liście `library`. Wypisz tytuł i autora każdej książki, iterując po liście.

13. Napisz funkcję `analyze_numbers(data)`, która przyjmuje krotkę liczb i zwraca krotkę trzech wartości: minimum, maksimum i średnią arytmetyczną. Wywołaj funkcję i rozpakuj wynik do trzech zmiennych `min_val`, `max_val`, `avg_val`.

14. Dana jest krotka:

```
original = (10, 20, 30, 40, 50)
```

Zamień krotkę na listę, dodaj element `60`, usuń element `20`, posortuj malejąco, a następnie zamień z powrotem na krotkę `modified`. Wypisz krotkę wejściową i wynikową.

15. Dana jest struktura przechowująca dane o uczniach:

```
classroom = (
    ("Alice", (90, 85, 92)),
    ("Bob", (78, 82, 88)),
    ("Charlie", (95, 91, 87))
)
```

Napisz kod, który wypisuje imię każdego ucznia wraz ze średnią jego ocen, korzystając wyłącznie z indeksowania i pętli `for`.

16. Dane są krotki:

```
version_a = (2, 8, 1)
version_b = (2, 8, 3)
version_c = (2, 10, 0)
version_d = (3, 0, 0)
```

Wypisz wyniki porównań `version_a < version_b`, `version_b < version_c` i `version_c < version_d`. W komentarzu wyjaśnij zasadę leksykograficznego porównywania krotek.

17. Utwórz listę krotek reprezentujących współrzędne punktów:

```
raw_points = [(1, 2), (3, 4), (1, 2), (5, 6), (3, 4), (7, 8)]
```

Zamień listę na `set`, aby usunąć duplikaty, a następnie zamień wynik z powrotem na posortowaną listę krotek `unique_points`. Wypisz wynik i wyjaśnij komentarzem, dlaczego krotki mogą być elementami zbioru, a listy nie.

18. Dana jest lista słowników:

```
sensors = [
    {"id": "S1", "temp": 22.5, "humidity": 45},
    {"id": "S2", "temp": 19.8, "humidity": 60},
    {"id": "S3", "temp": 25.1, "humidity": 38},
    {"id": "S4", "temp": 18.3, "humidity": 72},
]
```

Za pomocą generator expression i `tuple()` utwórz krotkę `high_humidity` zawierającą krotki `(id, humidity)` tylko dla czujników, których wilgotność przekracza 50.

19. Zaimportuj moduł `sys`. Utwórz listę `data_list` i krotkę `data_tuple` zawierające te same 1000 liczb całkowitych (od 0 do 999). Porównaj rozmiar obu struktur za pomocą `sys.getsizeof()`. Wypisz wyniki i wyjaśnij komentarzem, dlaczego krotka zajmuje mniej pamięci.
20. Napisz trzy funkcje operujące wyłącznie na krotkach (bez konwersji na listę):
21. `tuple_replace(data, index, value)` — zwraca nową krotkę, w której element o podanym indeksie został zastąpiony wartością `value` (wykorzystaj slicing i konkatenację krotek).
22. `tuple_insert(data, index, value)` — zwraca nową krotkę z wstawionym elementem na podanej pozycji.
23. `tuple_remove(data, index)` — zwraca nową krotkę bez elementu o podanym indeksie.

Przetestuj każdą funkcję na krotce `sample = (10, 20, 30, 40, 50)` i wypisz wyniki.

7 Zbiory

1. Utwórz zbiór `colors` zawierający pięć nazw kolorów. Wypisz zbiór oraz jego liczność (`len()`). Uruchom program kilka razy i sprawdź, czy kolejność wypisywanych elementów jest zawsze taka sama jak w definicji.
2. Utwórz pusty zbiór `visited`. Sprawdź typ wyrażenia `{}` i wyjaśnij w komentarzu, dlaczego pusty zbiór trzeba utworzyć za pomocą `set()`. Następnie dodaj do zbioru trzy nazwy miast metodą `.add()`, w tym jedno miasto dwukrotnie. Wypisz wynik.
3. Dana jest lista:

```
numbers = [3, 7, 3, 1, 9, 7, 7, 2, 1, 3]
```

Za pomocą zbioru wypisz liczbę różnych wartości występujących w liście oraz posortowaną listę tych wartości (`sorted()`).

4. Dany jest zbiór `fruits = {"apple", "banana", "cherry"}`. Usuń z niego element `"banana"` metodą `.remove()`, a następnie spróbuj usunąć element `"mango"` najpierw metodą `.remove()`, potem `.discard()`. Opisz w komentarzu różnicę między tymi metodami.
5. Wczytaj od użytkownika słowo i sprawdź za pomocą operatora `in`, czy należy do zbioru słów zarezerwowanych:

```
keywords = {"if", "else", "elif", "for", "while", "def", "return", "import", "class"}
```

6. Dane są zbiory:

```
python_students = {"Alice", "Bob", "Charlie", "Diana", "Eve"}
java_students = {"Charlie", "Eve", "Frank", "Grace"}
```

Wypisz (używając operatorów `|`, `&`, `-`, `^`):

- studentów uczących się co najmniej jednego języka,
- studentów uczących się obu języków,
- studentów uczących się tylko Pythona,
- studentów uczących się dokładnie jednego języka.

Następnie zapisz te same operacje za pomocą metod `.union()`, `.intersection()`, `.difference()` i `.symmetric_difference()`.

7. Wczytaj od użytkownika dwa napisy i wypisz:
   - litery występujące w obu napisach,
   - litery występujące tylko w pierwszym napisie,
   - liczbę różnych liter użytych łącznie w obu napisach.
8. Wczytaj od użytkownika zdanie i sprawdź, czy jest pangramem, tzn. czy zawiera wszystkie litery alfabetu angielskiego (`"abcdefghijklmnopqrstuvwxyz"`). Wielkość liter nie ma znaczenia. Jeśli zdanie nie jest pangramem, wypisz brakujące litery w kolejności alfabetycznej.
9. Dane są zbiory:

```
required_skills = {"python", "sql", "git"}
candidate_a = {"python", "sql", "git", "docker"}
candidate_b = {"python", "excel"}
```

Za pomocą metod `.issubset()` i `.issuperset()` (lub operatorów `<=`, `>=`) sprawdź, który kandydat posiada wszystkie wymagane umiejętności. Dla kandydata, który ich nie posiada, wypisz brakujące umiejętności. Sprawdź również metodą `.isdisjoint()`, czy kandydat B ma jakiekolwiek wymagane umiejętności.

10. Za pomocą set comprehension utwórz zbiór reszt z dzielenia przez 7 kwadratów liczb od 1 do 50. Wypisz posortowany wynik.
11. Dana jest lista słów:

```
words = ["Python", "java", "python", "C", "JAVA", "Rust", "c", "rust", "Go"]
```

Utwórz zbiór `unique_languages` zawierający nazwy języków bez powtórzeń, ignorując wielkość liter (zapisane małymi literami). Wypisz liczbę różnych języków.

12. Dana jest lista:

```
raw_data = [5, 3, 5, 1, 3, 8, 1, 9, 8]
```

Utwórz listę `unique_data` bez duplikatów, zachowując kolejność pierwszego wystąpienia. Do sprawdzania, czy element już wystąpił, użyj pomocniczego zbioru `seen`.

13. Dana jest lista:

```
numbers = [4, 9, 1, 4, 7, 9, 2, 4, 1]
```

Za pomocą dwóch zbiorów (`seen` i `duplicates`) i jednej pętli wyznacz zbiór wartości, które występują w liście więcej niż raz.

14. Dane są dwie listy identyfikatorów zamówień:

```
orders_january = [101, 102, 103, 104, 105, 106]
orders_february = [104, 105, 106, 107, 108]
```

Wypisz zamówienia złożone tylko w styczniu, tylko w lutym oraz w obu miesiącach. Wynik każdej operacji wypisz jako posortowaną listę.

15. Utwórz zbiór krotek reprezentujących współrzędne pól odwiedzonych przez robota:

```
moves = ["R", "R", "U", "L", "D", "D", "R", "U", "U"]
```

Robot startuje z pola `(0, 0)`; ruch `R` zwiększa `x` o 1, `L` zmniejsza `x`, `U` zwiększa `y`, `D` zmniejsza `y`. Wypisz, ile różnych pól odwiedził robot (łącznie z polem startowym) oraz czy któreś pole odwiedził więcej niż raz.

16. Utwórz zbiór `frozen_primes = frozenset({2, 3, 5, 7})`. Sprawdź, czy da się dodać do niego element. Następnie utwórz zbiór zbiorów, którego elementami są obiekty `frozenset`, i wyjaśnij w komentarzu, dlaczego elementami zbioru nie mogą być zwykłe zbiory ani listy.

17. Wczytaj od użytkownika liczbę naturalną `n` i wyznacz zbiór liczb pierwszych mniejszych od `n` metodą sita Eratostenesa, przechowując liczby złożone w zbiorze `composite`. Wynik wypisz jako posortowaną listę.

18. Dla `n = 100 000` utwórz listę i zbiór zawierające liczby od 0 do `n - 1`. Za pomocą modułu `time` (`time.perf_counter()`) zmierz łączny czas wykonania 1000 sprawdzeń przynależności (`in`) dla liczby `n - 1` w liście i w zbiorze. Wypisz wyniki i w komentarzu wyjaśnij różnicę.

8 Słowniki

1. Utwórz słownik `student` zawierający klucze: `name`, `age`, `grade`. Przypisz im dowolne wartości, a następnie wypisz wartość przypisaną do klucza `name`.
2. Dany jest słownik `fruits = {"apple": 3, "banana": 5, "cherry": 2}`. Dodaj nowy klucz `orange` z wartością 7, a następnie wypisz cały słownik.
3. Dany jest słownik `config = {"host": "localhost", "port": 8080, "debug": True}`. Usuń klucz `debug` za pomocą metody `pop()` i wypisz usuniętą wartość oraz pozostały słownik.
4. Dany jest słownik `prices = {"milk": 3.50, "bread": 2.00, "eggs": 5.99, "butter": 4.25}`. Napisz pętlę, która wypisze tylko te produkty, których cena jest wyższa niż 3.00.
5. Utwórz słownik `squares` za pomocą dict comprehension, który dla liczb od 1 do 10 przypisze każdej liczbie jej kwadrat (np. `{1: 1, 2: 4, 3: 9, ...}`).
6. Dane są dwie listy: `keys = ["name", "city", "language"]` oraz `values = ["Alice", "Warsaw", "Python"]`. Utwórz z nich słownik `profile` za pomocą funkcji `zip()`.
7. Dany jest słownik `inventory = {"apples": 10, "bananas": 0, "cherries": 5, "dates": 0}`. Za pomocą dict comprehension utwórz nowy słownik `in_stock`, który zawiera tylko produkty z ilością większą niż 0.
8. Dany jest słownik `scores = {"Alice": 85, "Bob": 92, "Charlie": 78, "Diana": 95}`. Znajdź klucz z najwyższą wartością, korzystając z funkcji `max()` i parametru `key`.
9. Napisz funkcję `merge_dicts(dict_a, dict_b)`, która łączy dwa słowniki. Jeśli oba słowniki mają ten sam klucz, wartości powinny zostać zsumowane. Zwróć nowy słownik.
10. Dany jest tekst `text = "apple banana apple cherry banana apple"`. Napisz kod, który zliczy wystąpienia każdego słowa i zapisze wynik w słowniku `word_count` bez użycia klasy `Counter`.
11. Dany jest słownik `data = {"a": 1, "b": 2, "c": 3}`. Napisz kod, który zamieni miejscami klucze z wartościami, tworząc nowy słownik `inverted_data` (np. `{1: "a", 2: "b", 3: "c"}`).

12. Utwórz zagnieżdżony słownik `school`, który przechowuje dane trzech uczniów. Każdy uczeń (klucz to `student_1`, `student_2`, `student_3`) ma przypisany słownik z kluczami `name`, `age` i `grades` (lista ocen). Wypisz średnią ocen drugiego ucznia.

13. Napisz funkcję `group_by_length(words)`, która przyjmuje listę stringów i zwraca słownik, w którym kluczem jest długość słowa, a wartością lista słów o tej długości. Użyj metody `setdefault()`.

14. Dany jest słownik `settings = {"resolution": "1080p", "volume": 75, "brightness": 50}`. Napisz funkcję `update_settings(current, overrides)`, która aktualizuje wartości w `current` tylko dla kluczy, które już w nim istnieją, ignorując nowe klucze z `overrides`.

15. Dany jest słownik `nested = {"a": {"b": {"c": 42}}}`. Napisz funkcję `deep_get(dictionary, keys_path, default_value=None)`, która przyjmuje słownik i string z kluczami rozdzielonymi kropką (np. `"a.b.c"`) i zwraca wartość zagnieżdżoną lub `default_value`, jeśli ścieżka nie istnieje.

16. Napisz funkcję `flatten_dict(nested_dict, parent_key="", separator=".")`, która spłaszcza zagnieżdżony słownik do jednopoziomowego. Na przykład `{"a": {"b": 1, "c": {"d": 2}}}` powinno dać `{"a.b": 1, "a.c.d": 2}`. Użyj rekurencji.

17. Napisz funkcję `diff_dicts(dict_a, dict_b)`, która porównuje dwa słowniki i zwraca słownik z trzema kluczami: `added` (klucze obecne tylko w `dict_b`), `removed` (klucze obecne tylko w `dict_a`) oraz `changed` (klucze obecne w obu, ale z różnymi wartościami, zapisane jako krotka `(old_value, new_value)`).

9 Funkcje

1. Napisz funkcję `print_header()`, która nie przyjmuje argumentów i wypisuje na ekranie ramkę z napisem `Wstęp do programowania`. Wywołaj ją dwukrotnie.

2. Napisz funkcję `square(x)`, która zwraca kwadrat liczby `x`. Wywołaj ją dla kilku wartości i wypisz wyniki. Następnie wykorzystaj ją do obliczenia sumy kwadratów liczb od 1 do 10.

3. Napisz funkcję `say_hello(name)`, która tylko wypisuje powitanie (bez instrukcji `return`). Przypisz wynik jej wywołania do zmiennej `result` i wypisz tę zmienną. W komentarzu wyjaśnij, skąd wzięła się wartość `None`.

4. Napisz funkcję `is_even(n)`, która zwraca `True`, jeśli liczba jest parzysta, oraz `False` w przeciwnym wypadku. Zapisz ciało funkcji w jednej linii, zwracając bezpośrednio wynik porównania. Użyj tej funkcji w pętli do wypisania liczb parzystych z zakresu od 1 do 20.

5. Napisz funkcję `max_of_three(a, b, c)`, która zwraca największą z trzech liczb bez użycia funkcji `max()`. Dodaj do funkcji docstring opisujący jej działanie i wypisz go za pomocą `help(max_of_three)`.

6. Napisz funkcję `is_prime(n)`, która sprawdza, czy liczba jest pierwsza. Zakończ działanie funkcji instrukcją `return` od razu po znalezieniu dzielnika. Wykorzystaj ją do wypisania wszystkich liczb pierwszych mniejszych od 100.

7. Napisz funkcję `min_max_avg(numbers)`, która przyjmuje listę liczb i zwraca jednocześnie minimum, maksimum i średnią (jako krotkę). Wywołaj ją i rozpakuj wynik do trzech zmiennych.

8. Napisz funkcję `count_vowels(text)`, która zwraca liczbę samogłosek w napisie, oraz funkcję `count_consonants(text)`, która zwraca liczbę spółgłosek (uwzględnij tylko litery). Przetestuj obie funkcje na kilku napisach.

9. Napisz funkcję `power(base, exponent=2)`, w której wykładnik ma wartość domyślną. Wywołaj ją z jednym i z dwoma argumentami.

10. Napisz funkcję `format_price(amount, currency="PLN", decimals=2)`, która zwraca napis z kwotą zaokrągloną do podanej liczby miejsc po przecinku i z oznaczeniem waluty. Wywołaj ją na kilka sposobów, korzystając z wartości domyślnych.

11. Napisz funkcję `add_item(item, items=[])`, która dodaje element do listy i ją zwraca. Wywołaj ją trzykrotnie bez podawania drugiego argumentu i wypisz wyniki. Wyjaśnij w komentarzu, dlaczego wynik jest zaskakujący, a następnie popraw funkcję, używając wartości domyślnej `None`.

12. Przeanalizuj poniższy kod, przewidź, co zostanie wypisane, a następnie uruchom go i sprawdź swoją odpowiedź:

```
x = 10

def show():
    x = 5
    print("wewnątrz:", x)

show()
print("na zewnątrz:", x)
```

W komentarzu wyjaśnij pojęcia zmiennej lokalnej i globalnej.

13. Sprawdź, co się stanie po uruchomieniu poniższego kodu, i wyjaśnij przyczynę błędu:

```
counter = 0

def increment():
    counter = counter + 1

increment()
```

Popraw kod na dwa sposoby: z użyciem słowa kluczowego `global` oraz bez niego (funkcja przyjmuje wartość jako argument i zwraca nową). W komentarzu wskaż, które rozwiązanie jest lepsze i dlaczego.

14. Utwórz globalny słownik `settings = {"theme": "light", "language": "pl"}` oraz funkcję `set_option(key, value)`, która zmienia w nim wartość. Wyjaśnij w komentarzu, dlaczego w tym przypadku nie jest potrzebne słowo kluczowe `global`, a w zadaniu 13 było.

15. Napisz funkcję `print_report(scores)`, która przyjmuje słownik `{imię: punkty}` i wypisuje raport. Wewnątrz zdefiniuj funkcję zagnieżdżoną `grade(points)`, zwracającą ocenę na podstawie liczby punktów (skala dowolna), i wykorzystaj ją przy wypisywaniu każdego wiersza raportu. Sprawdź, czy funkcję `grade` można wywołać spoza `print_report`.

16. Napisz funkcję `make_multiplier(factor)`, która zwraca funkcję zagnieżdżoną mnożącą swój argument przez `factor`. Utwórz za jej pomocą funkcje `double` i `triple` i przetestuj je.

17. Napisz funkcję `make_counter()`, która zwraca funkcję zagnieżdżoną zwiększającą przy każdym wywołaniu wewnętrzny licznik o 1 i zwracającą jego aktualną wartość. Wykorzystaj słowo kluczowe `nonlocal`. Pokaż, że dwa liczniki utworzone przez `make_counter()` działają niezależnie.

18. Napisz rekurencyjną funkcję `factorial(n)` obliczającą silnię oraz jej wersję iteracyjną `factorial_iter(n)`. Porównaj wyniki dla `n` od 0 do 10.

19. Napisz rekurencyjną funkcję `fibonacci(n)` zwracającą `n`-ty wyraz ciągu Fibonacciego. Dodaj globalny licznik wywołań i sprawdź, ile razy funkcja zostaje wywołana dla `n = 10`, `n = 20` i `n = 25`. W komentarzu wyjaśnij, skąd bierze się tak duża liczba wywołań.

20. Napisz rekurencyjną funkcję `sum_digits(n)` zwracającą sumę cyfr liczby naturalnej oraz rekurencyjną funkcję `is_palindrome(text)`, która sprawdza, czy napis jest palindromem (porównując pierwszy i ostatni znak oraz wywołując się dla napisu bez nich).

21. Napisz program do zarządzania listą zadań (to-do), podzielony na funkcje: `add_task(tasks, title)`, `remove_task(tasks, index)`, `show_tasks(tasks)` oraz `main()`, która w pętli wyświetla menu, wczytuje wybór użytkownika i wywołuje odpowiednie funkcje. Lista zadań ma być tworzona w funkcji `main()` i przekazywana do pozostałych funkcji jako argument — nie używaj zmiennych globalnych.

10 Argumenty funkcji

1. Napisz funkcję `add_numbers(a, b)`, która przyjmuje dwa argumenty pozycyjne typu int i zwraca ich sumę. Wywołaj ją z argumentami 3 i 5.
2. Napisz funkcję `greet(name, greeting="Hello")`, która przyjmuje imię jako argument obowiązkowy oraz powitanie jako argument z wartością domyślną. Funkcja ma zwracać sformatowany tekst powitania. Wywołaj ją na dwa sposoby — z jednym i z dwoma argumentami.
3. Napisz funkcję `describe_pet(animal_type, pet_name)` i wywołaj ją, przekazując oba argumenty jako argumenty kluczowe (keyword arguments) w odwrotnej kolejności niż w definicji funkcji.
4. Napisz funkcję `make_sandwich(*ingredients)`, która przyjmuje dowolną liczbę argumentów pozycyjnych reprezentujących składniki kanapki i zwraca je jako jedną listę. Wywołaj ją z trzema, a potem z sześcioma argumentami.
5. Napisz funkcję `build_profile(**user_info)`, która przyjmuje dowolną liczbę argumentów kluczowych i zwraca je w postaci słownika. Wywołaj ją, przekazując co najmniej cztery różne informacje o użytkowniku.
6. Napisz funkcję `calculate_power(base, exponent=2)`, w której argument `exponent` ma wartość domyślną 2. Wywołaj ją trzykrotnie — bez podawania wykładnika, z wykładnikiem 3 oraz z wykładnikiem przekazanym jako argument kluczowy.
7. Napisz funkcję `format_address(street, city, zip_code, country="Poland")`, która łączy dane adresowe w jeden sformatowany ciąg znaków. Wywołaj ją, mieszając argumenty pozycyjne i kluczowe w jednym wywołaniu.
8. Napisz funkcję `apply_discount(price, discount_percent, /, tax_rate=0.23)`, która używa operatora `/` wymuszającego, aby `price` i `discount_percent` były wyłącznie argumentami pozycyjnymi. Pokaż, że próba przekazania ich jako keyword arguments kończy się błędem.
9. Napisz funkcję `register_user(username, *, email, age)`, która używa operatora `*` wymuszającego, aby `email` i `age` były przekazywane wyłącznie jako argumenty kluczowe. Wywołaj ją poprawnie, a następnie pokaż błędne wywołanie z samymi argumentami pozycyjnymi.

10. Napisz funkcję `mix_paint(primary, secondary, /, *, finish="matte")`, która łączy oba mechanizmy — argumenty wyłącznie pozycyjne (przed `/`) i wyłącznie kluczowe (po `*`). Wywołaj ją na trzy różne sposoby.

11. Napisz funkcję `log_message(message, *tags, level="INFO")`, która przyjmuje jeden argument obowiązkowy, dowolną liczbę tagów przez `*args` oraz argument kluczowy z wartością domyślną na końcu. Wywołaj ją z dwoma tagami i zmienionym poziomem logowania.

12. Napisz funkcję `create_report(title, *sections, **metadata)`, która łączy argument obowiązkowy, `*args` i `**kwargs` w jednej sygnaturze. Wywołaj ją, przekazując tytuł, trzy sekcje oraz dwie pary klucz-wartość jako metadane.

13. Napisz dwie funkcje — `square(n)` zwracającą kwadrat liczby oraz `double(n)` zwracającą jej podwojenie. Następnie napisz funkcję `apply_operation(func, value)`, która przyjmuje inną funkcję jako argument i wywołuje ją na przekazanej wartości. Przetestuj ją z oboma funkcjami.

14. Napisz funkcję `process_data(data, *, transformer=None, validator=None)`, w której oba argumenty opcjonalne to funkcje (lub None). Jeśli `transformer` nie jest None, funkcja ma przetworzyć dane, a jeśli `validator` nie jest None — zwalidować wynik. Wywołaj ją z funkcją lambda jako transformerem.

15. Napisz funkcję `repeat_action(action, times=1, /, *, delay=0)`, a następnie wywołaj ją, przekazując jako argument `action` funkcję lambda wypisującą tekst. Pokaż trzy wywołania z różnymi kombinacjami argumentów pozycyjnych i kluczowych.

16. Napisz funkcję `merge_configs(*config_dicts, **overrides)`, która przyjmuje dowolną liczbę słowników jako `*args`, łączy je w jeden, a potem nadpisuje wartości przekazane jako `**kwargs`. Przetestuj ją z trzema słownikami konfiguracyjnymi i dwoma nadpisaniami.

17. Napisz funkcję `safe_divide(numerator, denominator, /, *, on_error="return_none")`, która dzieli dwie liczby, a w przypadku dzielenia przez zero zachowuje się zgodnie z wartością argumentu `on_error` — może zwrócić None, zero albo rzucić wyjątek. Przetestuj każdy z trzech trybów.

18. Napisz funkcję `pipeline(value, *transformations)`, która przyjmuje wartość początkową i dowolną liczbę funkcji. Funkcja ma zastosować każdą z nich po kolei na wyniku poprzedniej. Przetestuj ją, przekazując trzy funkcje lambda tworzące łańcuch przekształceń.

19. Napisz dekorator `validate_types(*expected_types)`, który przyjmuje przez `*args` oczekiwane typy kolejnych argumentów dekorowanej funkcji. Dekorator ma przed wywołaniem funkcji sprawdzić, czy typy przekazanych argumentów zgadzają się z oczekiwanymi, i w razie niezgodności rzucić TypeError. Zastosuj go do funkcji `concat_strings(a, b)`.

20. Napisz funkcję `create_dispatcher()`, która zwraca dwie funkcje wewnętrzne — `register(name, func)` do rejestrowania funkcji pod podaną nazwą oraz `call(name, *args, **kwargs)` do wywoływania zarejestrowanej funkcji z dowolnymi argumentami. Przetestuj mechanizm, rejestrując trzy funkcje o różnych sygnaturach i wywołując je przez dispatcher z różnymi kombinacjami argumentów pozycyjnych, kluczowych i domyślnych.

11 f-stringi

1. Zdefiniuj zmienną `name = "Anna"` i zmienną `age = 28`, a następnie wypisz je w jednym zdaniu za pomocą f-stringa w postaci `f"Imię: {name}, wiek: {age}"`.
2. Mając zmienną `age = 28`, napisz f-string, który wewnątrz klamer obliczy wartość wyrażenia `age + 5` i wypisze wynik w zdaniu `"Za 5 lat będziesz mieć: ..."`.
3. Zdefiniuj zmienną `name = "Anna"` i napisz f-string, który wewnątrz klamer wywoła metodę `name.upper()`, wypisując imię wielkimi literami.
4. Mając zmienną `pi = 3.141592653589793`, napisz f-string ze specyfikatorem `:.4f`, który wyświetli wartość pi zaokrągloną do czterech miejsc po przecinku.
5. Zdefiniuj zmienną `loss = 0.00034521` i użyj f-stringa ze specyfikatorem `:.2e`, aby wypisać tę wartość w notacji naukowej z dwoma miejscami po przecinku.
6. Mając zmienną `accuracy = 0.9534`, napisz f-string ze specyfikatorem `:.1%`, który wyświetli dokładność jako wartość procentową z jednym miejscem po przecinku.
7. Zdefiniuj zmienną `population = 38179800` i napisz f-string z separatorem tysięcy `:,`, wypisując wartość w czytelnym formacie z przecinkami.
8. Mając zmienną `revenue = 1543267.89`, napisz f-string, który połączy separator tysięcy z precyzją dwóch miejsc po przecinku, stosując specyfikator `:,.2f`, i doda na końcu walutę `PLN`.
9. Zdefiniuj zmienną `population = 38179800` i użyj f-stringa ze specyfikatorem `:_`, aby wypisać wartość z podkreślnikami jako separatorami tysięcy.
10. Mając listę krotek `metrics = [("Accuracy", 0.9534), ("Precision", 0.9211), ("Recall", 0.8876)]`, napisz pętlę `for`, która za pomocą f-stringa z wyrównaniem do lewej `:<12` wypisze nazwy metryk w kolumnie o szerokości 12 znaków, a obok ich wartości w formacie `:.2%`.
11. Wykorzystując tę samą listę `metrics`, napisz pętlę `for` z f-stringiem, w którym zastosujesz wyrównanie do prawej `:>15` z gwiazdką `*` jako znakiem wypełnienia, a po dwukropku wartość w formacie `:.4f`.
12. Mając zmienną `x = 42`, napisz f-string ze specyfikatorem `:^10`, który wycentruje liczbę w polu o szerokości 10 znaków. Opisz, jaką rolę pełni tu operator `^`.

13. Zaimportuj `datetime` i zdefiniuj `now = datetime.now()`. Napisz f-string z formatem `:%Y-%m-%d`, który wypisze bieżącą datę w standardzie ISO.

14. Mając tę samą zmienną `now`, napisz dwa f-stringi: jeden z formatem `:%H:%M:%S` wypisujący godzinę, drugi z formatem `:%d %B %Y` wypisujący datę w formacie europejskim ze słowną nazwą miesiąca.

15. Zdefiniuj zmienne `learning_rate = 0.001`, `epochs = 50` i `batch_size = 32`. Użyj operatora debugowania `=` w f-stringu (np. `f"{learning_rate = }"`) do wypisania nazwy i wartości każdej zmiennej.

16. Mając zmienną `loss = 0.0023456`, napisz f-string łączący operator `=` z formatowaniem, w postaci `f"{loss = :.4f}"`, i opisz, co zostanie wypisane.

17. Mając zmienną `train_acc = 0.9812`, napisz f-string, który za pomocą operatora `=` i specyfikatora `:.1%` wypisze jednocześnie nazwę zmiennej i jej wartość procentową.

18. Zdefiniuj listę `values = [10, 20, 30, 40]` i napisz f-string, w którym wewnątrz klamer wywołasz wyrażenie `sum(values) / len(values)`, wypisując średnią arytmetyczną.

19. Mając zmienne `model_name = "RandomForestClassifier"`, `accuracy = 0.9534`, `precision = 0.9211`, `recall = 0.8876` i `n_samples = 150000`, napisz wieloliniowy f-string (w potrójnych cudzysłowach), który sformatuje raport z wynikami modelu. Użyj linii z `{'='*50}` jako separatorów, wyrównania do prawej `:>8.2%` dla metryk i separatora tysięcy `:,` dla liczby próbek.

20. Mając zmienne z poprzedniego zadania, oblicz dodatkowo `f1 = 2 * precision * recall / (precision + recall)` i rozbuduj wieloliniowy f-string o wiersz z F1-score. Zadbaj o to, aby wszystkie metryki miały jednolite wyrównanie do prawej w kolumnie o szerokości 8 znaków ze specyfikatorem `:.2%`, a cały raport był otoczony dekoracyjnymi liniami wygenerowanymi wyrażeniem w klamrach.

12 Obsługa wyjątków

1. Uruchom kolejno poniższe instrukcje (każdą osobno) i zapisz w komentarzu nazwę zgłoszonego wyjątku:

```
10 / 0
int("abc")
[1, 2, 3][5]
{"a": 1}["b"]
"5" + 5
undefined_variable
```

2. Wczytaj od użytkownika liczbę całkowitą. Jeśli użytkownik poda wartość, której nie da się przekonwertować, przechwyć wyjątek `ValueError` i wypisz czytelny komunikat zamiast komunikatu błędu interpretera.
3. Wczytaj od użytkownika dwie liczby i wypisz wynik ich dzielenia. Obsłuż osobnymi blokami `except` wyjątki `ValueError` (błędne dane) oraz `ZeroDivisionError` (dzielenie przez zero), wypisując dla każdego inny komunikat.
4. Zmodyfikuj program z poprzedniego zadania tak, aby wynik dzielenia był wypisywany w bloku `else`, a w bloku `finally` wypisywany był komunikat `"Koniec obliczeń"`. Sprawdź, kiedy wykonuje się blok `else`, a kiedy `finally`.
5. Przechwyć wyjątek za pomocą konstrukcji `except ValueError as error:` i wypisz treść komunikatu (`error`) oraz typ wyjątku (`type(error).__name__`).
6. Napisz funkcję `read_int(prompt)`, która w pętli wczytuje dane od użytkownika tak długo, aż poda on poprawną liczbę całkowitą, i zwraca tę liczbę. Wykorzystaj ją do wczytania wieku użytkownika.
7. Napisz funkcję `read_int_in_range(prompt, low, high)`, która działa jak funkcja z poprzedniego zadania, ale dodatkowo wymaga, aby liczba należała do przedziału `[low, high]`. Przetestuj ją na wczytywaniu oceny z zakresu od 2 do 5.
8. Dana jest lista:

```
data = ["12", "7", "abc", "3.5", "", "40", "-8"]
```

Za pomocą pętli i bloku `try/except` utwórz listę `numbers` zawierającą tylko te elementy, które dają się przekonwertować na `int`, oraz listę `errors` zawierającą pozostałe elementy.

9. Dany jest słownik:

```
stock = {"apple": 10, "banana": 0, "cherry": 25}
```

Napisz funkcję `get_stock(product)`, która zwraca liczbę sztuk produktu, a w przypadku nieistniejącego klucza przechwytuje `KeyError` i zwraca `0`. Następnie zapisz drugą wersję funkcji bez obsługi wyjątków, z użyciem metody `.get()`. W komentarzu porównaj oba podejścia.

10. Napisz funkcję `safe_get(items, index, default=None)`, która zwraca element listy o podanym indeksie, a w przypadku wyjątku `IndexError` zwraca wartość domyślną.
11. Napisz funkcję `calculate_average(numbers)`, która zgłasza wyjątek `ValueError` z komunikatem `"Lista nie może być pusta"` (instrukcja `raise`), gdy przekazano pustą listę, oraz `TypeError`, gdy któryś element nie jest liczbą (wykorzystaj `isinstance(x, (int, float))`). Przetestuj funkcję w bloku `try/except`.
12. Napisz funkcję `set_age(age)`, która zgłasza `ValueError`, jeśli wiek jest ujemny lub większy niż 150, a w przeciwnym razie zwraca go. Przetestuj ją w pętli dla listy wartości `[25, -3, 200, 47]`, wypisując dla każdej wartości wynik albo komunikat błędu.
13. Obsłuż kilka wyjątków w jednym bloku: `except (ValueError, ZeroDivisionError) as error:`. Napisz program, który wczytuje dwie liczby i wypisuje wynik dzielenia, a w przypadku dowolnego z tych dwóch błędów wypisuje jeden wspólny komunikat zawierający treść wyjątku.
14. Wyjaśnij w komentarzu, dlaczego konstrukcja `except:` (bez nazwy wyjątku) lub `except Exception:` obejmująca duży fragment kodu jest uważana za złą praktykę. Pokaż przykład, w którym taki blok ukrywa błąd programisty (np. literówkę w nazwie zmiennej).
15. Napisz funkcję `parse_point(text)`, która przyjmuje napis w postaci `"x,y"` (np. `"3,4"`) i zwraca krotkę dwóch liczb zmiennoprzecinkowych. Jeśli napis ma złą postać, funkcja powinna przechwycić wyjątek, a następnie zgłosić nowy `ValueError` z czytelnym komunikatem, łącząc go z oryginalnym wyjątkiem za pomocą `raise ... from error`.
16. Dana jest funkcja:

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Kwota musi być dodatnia")
    if amount > balance:
        raise ValueError("Brak wystarczających środków")
    return balance - amount
```

Napisz program, który w pętli wczytuje kwoty wypłat dla początkowego salda 1000 zł i wywołuje funkcję `withdraw`. Program obsługuje błędy zarówno przy konwersji danych, jak i zgłaszane przez funkcję, i kończy działanie po wpisaniu `q`.

17. Napisz funkcję `divide(a, b)`, w której na początku za pomocą instrukcji `assert` sprawdzisz, że `b != 0`, z komunikatem błędu. Wywołaj ją z błędnymi danymi i sprawdź, jaki wyjątek zostanie zgłoszony. W komentarzu wyjaśnij, dlaczego `assert` nie powinien zastępować walidacji danych od użytkownika.

18. Przeanalizuj poniższy kod i przewidź kolejność wypisywanych komunikatów dla wywołań `test(1)`, `test(0)` i `test("a")`. Sprawdź swoją odpowiedź, uruchamiając kod.

```
def test(x):
    try:
        print("try")
        result = 10 / x
    except ZeroDivisionError:
        print("except ZeroDivisionError")
        return "błąd"
    else:
        print("else")
        return result
    finally:
        print("finally")
```

Co dzieje się z wyjątkiem `TypeError` w wywołaniu `test("a")`?

19. Napisz kalkulator wyrażeń postaci `liczba operator liczba` (np. `12 * 4`), wczytywanych od użytkownika w pętli do momentu wpisania `koniec`. Wykorzystaj metodę `.split()`. Obsłuż wyjątki: błędną liczbę elementów wyrażenia, niepoprawne liczby, dzielenie przez zero i nieznany operator — każdy z odpowiednim komunikatem. Program nie może zakończyć się błędem niezależnie od wpisanych danych.

13 Obsługa plików

W zadaniach otwieraj pliki za pomocą instrukcji `with`, a w przypadku plików tekstowych jawnie podawaj kodowanie (`encoding="utf-8"`).

13.1 Pliki tekstowe

1. Utwórz plik `hello.txt` i zapisz w nim trzy linie tekstu (tryb `"w"`, metoda `.write()`). Pamiętaj o znakach nowej linii `"\n"`. Następnie otwórz plik w trybie `"r"`, wczytaj całą zawartość metodą `.read()` i wypisz ją.
2. Otwórz plik `hello.txt` z poprzedniego zadania w trybie `"a"` i dopisz na jego końcu dwie nowe linie. Wypisz zawartość pliku i sprawdź, czy poprzednie linie zostały zachowane. W komentarzu opisz różnicę między trybami `"w"`, `"a"` i `"x"`.
3. Wczytaj plik `hello.txt` linia po linii, iterując bezpośrednio po obiekcie pliku, i wypisz każdą linię poprzedzoną jej numerem. Usuń znaki nowej linii metodą `.rstrip("\n")`, aby uniknąć pustych linii przy wypisywaniu.
4. Porównaj działanie metod `.read()`, `.readline()` i `.readlines()` na tym samym pliku. Sprawdź typ zwracanego wyniku w każdym przypadku.
5. Wczytaj od użytkownika kolejne imiona (do podania pustego napisu) i zapisz je w pliku `names.txt` — każde w osobnej linii — za pomocą metody `.writelines()`.
6. Napisz funkcję `file_stats(path)`, która zwraca krotkę z liczbą linii, liczbą słów i liczbą znaków w pliku tekstowym (podobnie jak polecenie `wc` w systemach uniksowych). Przetestuj ją na wybranym pliku.
7. Napisz program, który próbuje otworzyć plik o nazwie podanej przez użytkownika i wypisuje jego zawartość. Jeśli plik nie istnieje, przechwyć wyjątek `FileNotFoundError` i wypisz odpowiedni komunikat.
8. Utwórz plik `numbers.txt` zawierający w kolejnych liniach liczby całkowite (część linii może być pusta lub zawierać tekst niebędący liczbą). Napisz program, który wczytuje plik, pomija niepoprawne linie (z obsługą wyjątku `ValueError`) i wypisuje sumę, minimum, maksimum oraz liczbę pominiętych linii.

9. Napisz funkcję `count_words(path)`, która zwraca słownik z liczbą wystąpień każdego słowa w pliku tekstowym. Zamień wszystkie litery na małe i usuń znaki interpunkcyjne `.,;:!?`. Wypisz 10 najczęściej występujących słów (wskazówka: `sorted(counts, key=counts.get, reverse=True)`).

10. Napisz funkcję `copy_uppercase(source, target)`, która kopiuje plik tekstowy `source` do pliku `target`, zamieniając wszystkie litery na wielkie. Otwórz oba pliki w jednej instrukcji `with`.

11. Dany jest plik `grades.csv` o zawartości:

```
name;math;physics;computer_science
Alice;5;4;5
Bob;3;4;4
Charlie;4;5;3
Diana;5;5;5
```

Wczytaj plik bez użycia dodatkowych modułów (metody `.strip()` i `.split(";")`), pomiń wiersz nagłówka i dla każdej osoby wypisz średnią ocen. Następnie zapisz do pliku `averages.csv` imiona i średnie (zaokrąglone do dwóch miejsc po przecinku).

12. Rozwiąż poprzednie zadanie ponownie, tym razem korzystając z modułu `csv` (`csv.reader` z parametrem `delimiter=";"` i `csv.writer`). Otwierając plik do zapisu, użyj parametru `newline=""`.

13. Napisz funkcję `save_contacts(contacts, path)`, która zapisuje słownik kontaktów `{imię: numer telefonu}` do pliku tekstowego w postaci `imię=numer` (każdy kontakt w osobnej linii), oraz funkcję `load_contacts(path)`, która odczytuje taki plik i zwraca słownik. Sprawdź, że po zapisie i odczycie otrzymasz słownik równy wyjściowemu.

14. Zapisz ten sam słownik kontaktów do pliku `contacts.json` za pomocą funkcji `json.dump()` (z parametrami `ensure_ascii=False` i `indent=2`) i odczytaj go funkcją `json.load()`. Otwórz plik w edytorze tekstu i porównaj jego zawartość z plikiem z poprzedniego zadania.

15. Napisz program dziennika: przy każdym uruchomieniu wczytuje od użytkownika jedną linię tekstu i dopisuje ją na końcu pliku `journal.txt` wraz z numerem kolejnego wpisu (numer wyznacz, licząc istniejące linie pliku; uwzględnij sytuację, gdy plik jeszcze nie istnieje).

13.2 Pliki binarne

16. Utwórz obiekt `bytes` za pomocą `bytes([72, 101, 108, 108, 111])` i wypisz go. Następnie zakoduj napis `"Zażółć gęślą jaźń"` do bajtów metodą `.encode("utf-8")`, wypisz liczbę znaków napisu i liczbę bajtów po zakodowaniu. W komentarzu wyjaśnij różnicę.

17. Zapisz napis `"Zażółć gęślą jaźń"` do pliku tekstowego z kodowaniem `utf-8`, a następnie otwórz ten plik w trybie binarnym `"rb"` i wypisz jego zawartość jako obiekt `bytes` oraz listę wartości kolejnych bajtów (`list(data)`). Spróbuj też odczytać plik w trybie tekstowym z kodowaniem `cp1250` i opisz wynik.

18. Zapisz do pliku binarnego `data.bin` (tryb `"wb"`) liczby od 0 do 255 jako kolejne bajty. Odczytaj plik, sprawdź jego rozmiar (`len()`) oraz wypisz bajty o indeksach od 65 do 90. Zdekoduj je do napisu i wyjaśnij wynik.

19. Zapisz do pliku binarnego liczby całkowite `[1, 300, 70000, -5]`, zamieniając każdą z nich na 4 bajty metodą `int.to_bytes(4, "little", signed=True)`. Następnie odczytaj plik po 4 bajty (`.read(4)`) i odtwórz liczby metodą `int.from_bytes()`.

20. Napisz funkcję `copy_binary(source, target, chunk_size=4096)`, która kopiuje dowolny plik (np. obraz JPG) w trybie binarnym, czytając go fragmentami o rozmiarze `chunk_size` w pętli `while`. Sprawdź, czy kopia jest identyczna z oryginałem, porównując zawartość obu plików odczytaną w trybie binarnym.

21. Pliki PNG zaczynają się od 8-bajtowej sygnatury `b"\x89PNG\r\n\x1a\n"`, a pliki JPEG od bajtów `b"\xff\xd8\xff"`. Napisz funkcję `detect_image_type(path)`, która odczytuje początek pliku i zwraca `"PNG"`, `"JPEG"` lub `"nieznany"`. Przetestuj ją na kilku plikach, w tym na pliku tekstowym.

22. Otwórz plik `data.bin` z zadania 18 w trybie `"rb"`. Za pomocą metody `.seek()` przejdź do bajtu o indeksie 100, odczytaj 5 bajtów i wypisz je. Następnie wypisz bieżącą pozycję w pliku metodą `.tell()`.

14 Struktura programu i moduły

1. Zaimportuj moduł `math` i wypisz wartość stałych `math.pi` i `math.e` oraz wyniki funkcji `math.sqrt(2)`, `math.floor(3.7)`, `math.ceil(3.2)` i `math.gcd(48, 18)`.

2. Zaimportuj z modułu `math` tylko funkcje `sqrt` i `hypot` (`from math import sqrt, hypot`) i oblicz długość przekątnej prostokąta o bokach 3 i 4 na dwa sposoby. W komentarzu opisz różnicę między `import math` a `from math import sqrt`.

3. Zaimportuj moduł `random` pod skróconą nazwą (`import random as rnd`). Wylosuj i wypisz: liczbę całkowitą z zakresu 1–6 (`randint`), liczbę zmiennoprzecinkową z przedziału $[0, 1)$ (`random`), losowy element listy (`choice`) oraz listę przetasowaną w miejscu (`shuffle`). Ustaw ziarno generatora (`seed`) i sprawdź, że wyniki stają się powtarzalne.

4. Za pomocą funkcji `dir()` wypisz nazwy dostępne w module `math`. Następnie za pomocą funkcji `help()` wyświetl dokumentację funkcji `math.isclose` i na jej podstawie porównaj wartości `0.1 + 0.2` oraz `0.3`.

5. Wykorzystując moduł `random`, napisz grę w zgadywanie liczby z rozdziału o pętlach, w której tajna liczba jest losowana przy każdym uruchomieniu programu.

6. Utwórz plik `geometry.py` zawierający funkcje `circle_area(r)`, `circle_perimeter(r)`, `rectangle_area(a, b)` oraz stałą `PI = 3.14159`. W tym samym katalogu utwórz plik `main.py`, który importuje moduł `geometry` i korzysta z jego funkcji. Uruchom `main.py`.

7. W pliku `main.py` z poprzedniego zadania zaimportuj z modułu `geometry` wybrane funkcje na trzy sposoby: `import geometry`, `import geometry as geo`, `from geometry import circle_area`. W komentarzu wyjaśnij, dlaczego zapis `from geometry import *` jest odradzany.

8. Do pliku `geometry.py` dodaj na końcu instrukcję `print("Załadowano moduł geometry")`. Uruchom `main.py` i sprawdź, kiedy pojawia się komunikat. Zaimportuj moduł dwukrotnie i sprawdź, ile razy zostanie wypisany komunikat.

9. W module `geometry.py` wypisz wartość zmiennej `__name__`. Sprawdź, jaką ma wartość, gdy uruchamiasz plik bezpośrednio, a jaką, gdy jest importowany z `main.py`. Następnie przenieś kod testujący funkcje modułu do bloku `if __name__ == "__main__":`, tak aby nie był wykonywany podczas importu.

10. Przekształć program w poniższej postaci tak, aby cały kod znajdował się w funkcjach, a program był uruchamiany przez funkcję `main()` wywoływaną w bloku `if __name__ == "__main__"`::

```
numbers = []
while True:
    text = input("Podaj liczbę (puste = koniec): ")
    if text == "":
        break
    numbers.append(float(text))
print("Suma:", sum(numbers))
print("Średnia:", sum(numbers) / len(numbers))
```

Wydziel osobne funkcje do wczytywania danych i obliczania statystyk. Zadbaj o obsługę pustej listy.

11. Napisz moduł `text_tools.py` z funkcjami `count_words(text)`, `reverse_words(text)` oraz `is_palindrome(text)` (ignorującą wielkość liter i spacje). Dodaj docstring na początku modułu i dla każdej funkcji. W osobnym pliku zaimportuj moduł i wywołaj `help(text_tools)`.

12. Utwórz pakiet `shapes` — katalog z plikiem `__init__.py` oraz modułami `circle.py` i `square.py`, z których każdy zawiera funkcje `area()` i `perimeter()`. W pliku `main.py` (poza katalogiem pakietu) zaimportuj moduły na dwa sposoby: `from shapes import circle` oraz `import shapes.square`.

13. W pliku `shapes/__init__.py` z poprzedniego zadania zaimportuj moduły `circle` i `square` (import względny: `from . import circle, square`), tak aby w `main.py` wystarczyło napisać `import shapes` i korzystać z `shapes.circle.area(...)`.

14. Zaimportuj moduł `sys` i wypisz listę `sys.path`. W komentarzu wyjaśnij, w jakiej kolejności Python wyszukuje importowane moduły i co się stanie, gdy utworzysz w katalogu projektu własny plik o nazwie `random.py`.

15. Napisz program `sum_args.py`, który sumuje liczby przekazane jako argumenty wiersza poleceń (`sys.argv`). Uruchom go poleceniem `python sum_args.py 3 4 5.5`. Obsłuż przypadek braku argumentów i argumentów, które nie są liczbami.

16. Program z rozdziału o funkcjach (lista zadań to-do) podziel na moduły:

    - `storage.py` — funkcje zapisu i odczytu listy zadań z pliku tekstowego,
    - `tasks.py` — funkcje dodawania, usuwania i wyświetlania zadań,
    - `main.py` — menu programu i funkcja `main()`.

Zadbaj o to, aby moduły `storage.py` i `tasks.py` nie korzystały z funkcji `input()` ani `print()` (z wyjątkiem funkcji wyświetlającej zadania) — cała komunikacja z użytkownikiem ma odbywać się w `main.py`.

17. Utwórz moduł `config.py` zawierający stałe konfiguracyjne (np. `FILE_NAME`, `MAX_TASKS`) zapisane wielkimi literami i wykorzystaj je w programie z poprzedniego zadania zamiast wartości wpisanych bezpośrednio w kod.

15 Zasady pisania czystego kodu

Zadania z tego rozdziału dotyczą zaleceń dokumentu PEP 8 oraz narzędzi wspierających pisanie czystego kodu. Narzędzia instaluje się poleceniem `pip install <nazwa>` (najlepiej w środowisku wirtualnym).

1. Popraw nazwy w poniższym kodzie zgodnie z konwencjami PEP 8 (`snake_case` dla zmiennych i funkcji, `UPPER_CASE` dla stałych). Nadaj nazwom znaczenie, które wyjaśnia rolę zmiennej.

```
def F(L):
    S = 0
    for X in L:
        S += X
    return S / len(L)

A = [3, 5, 4]
b = F(A)
print(b)
```

2. Popraw formatowanie poniższego kodu zgodnie z PEP 8 (wcięcia, spacje wokół operatorów i po przecinkach, puste linie, długość linii):

```
import math,random
def area(r):return math.pi*r**2
def main():
  for i in range(5):
    r=random.randint(1,10)
    print("r =",r,"pole =",round(area(r),2))
main()
```

3. Zainstaluj narzędzie `ruff` i uruchom `ruff check` na pliku z poprzedniego zadania (w wersji przed poprawkami). Przeanalizuj zgłoszone problemy, a następnie uruchom `ruff format` i porównaj wynik ze swoimi poprawkami.

4. Zainstaluj narzędzie `pylint` i uruchom je na wybranym rozwiązaniu z rozdziału o funkcjach. Popraw kod tak, aby uzyskać ocenę co najmniej 9/10. Zapisz w komentarzu, jakie rodzaje uwag pojawiły się najczęściej.

5. Zastąp „magiczne liczby” w poniższym kodzie stałymi o czytelnych nazwach:

```
def final_price(price):
    if price > 500:
        price = price * 0.9
    return price * 1.23
```

6. Dodaj docstringi (zgodne z PEP 257) do modułu oraz do każdej funkcji z zadania 5. Opisz w nich argumenty, zwracaną wartość oraz zgłaszane wyjątki. Wyświetl je za pomocą `help()`.

7. Dodaj adnotacje typów (type hints) do funkcji:

```
def average(numbers):
    return sum(numbers) / len(numbers)

def find_user(users, name):
    for user in users:
        if user["name"] == name:
            return user
    return None
```

Zainstaluj narzędzie `mypy` i sprawdź nim kod, w którym funkcja `average` zostaje wywołana z napisem zamiast listy.

8. Uprość poniższy kod, korzystając z wczesnego powrotu (`return`) zamiast zagnieżdżonych instrukcji warunkowych:

```
def can_vote(age, is_citizen, is_registered):
    if age >= 18:
        if is_citizen:
            if is_registered:
                return True
            else:
                return False
        else:
            return False
    else:
        return False
```

Następnie zapisz funkcję w jednej linii jako `return` z wyrażeniem logicznym.

9. Uprość poniższe fragmenty kodu, stosując idiomy języka Python:

```
if is_valid == True:
    print("OK")

if len(items) == 0:
    print("Pusto")

for i in range(len(names)):
    print(i, names[i])

result = []
for x in range(10):
    if x % 2 == 0:
        result.append(x * x)
```

10. W poniższym kodzie ten sam fragment powtarza się trzykrotnie. Zastosuj zasadę DRY (*Don't Repeat Yourself*), wydzielając funkcję:

```
math_scores = [4, 5, 3, 5]
print("Matematyka:", round(sum(math_scores) / len(math_scores), 2))
physics_scores = [3, 4, 4]
print("Fizyka:", round(sum(physics_scores) / len(physics_scores), 2))
cs_scores = [5, 5, 4, 5]
print("Informatyka:", round(sum(cs_scores) / len(cs_scores), 2))
```

11. Poniższa funkcja robi zbyt wiele rzeczy naraz. Podziel ją na mniejsze funkcje, z których każda ma jedno zadanie (wczytanie danych, obliczenia, wypisanie wyników):

```
def process():
    numbers = []
    while True:
        text = input("Liczba: ")
        if text == "":
            break
        numbers.append(int(text))
    even = [n for n in numbers if n % 2 == 0]
    odd = [n for n in numbers if n % 2 != 0]
    print("Parzyste:", even, "suma:", sum(even))
    print("Nieparzyste:", odd, "suma:", sum(odd))
```

12. Dodaj do funkcji `is_prime(n)` z rozdziału o funkcjach przykłady użycia w docstringu w formacie modułu `doctest` (linie zaczynające się od `>>>`). Uruchom testy poleceniem `python -m doctest -v nazwa_pliku.py`.

13. Napisz plik `test_geometry.py` z testami dla funkcji modułu `geometry.py` z rozdziału o modułach, korzystając z instrukcji `assert` w funkcjach o nazwach zaczynających się od `test_`. Zainstaluj narzędzie `pytest` i uruchom testy poleceniem `pytest`. Celowo wprowadź błąd do jednej z funkcji i sprawdź, jak `pytest` raportuje niepowodzenie.

14. Przeanalizuj poniższy kod i wypisz w komentarzu co najmniej pięć problemów związanych z czytelnością i jakością kodu. Następnie przepisz go w czystej postaci.

```
def calc(d):
    # d - dane
    r = {}
    for k in d:
        t = 0
        for i in d[k]:
            t = t + i
        r[k] = t / len(d[k]) if len(d[k]) != 0 else 0
    l = []
    for k in r:
        if r[k] >= 4.5: l.append(k)
    return r, l
```

15. Utwórz w katalogu projektu plik `pyproject.toml` z konfiguracją narzędzia `ruff` (np. maksymalna długość linii 88 znaków, włączone reguły `E`, `F`, `I`, `N`). Uruchom `ruff check` na całym projekcie z poprzednich rozdziałów i popraw zgłoszone problemy.

Część II

Programowanie obiektowe

16 Proste klasy

Dla każdej klasy należy poza klasą stworzyć obiekt co najmniej jeden raz.

1. Napisz klasę `Dog` z inicjalizatorem przyjmującym `name` i `age` oraz metodą `bark()`, która wypisuje `"<imię>: Hau!"`. Utwórz dwa obiekty tej klasy i wywołaj na każdym z nich metodę `bark()`. Wypisz również atrybuty obu obiektów.
2. Napisz klasę `Rectangle` z atrybutami `width` i `height` oraz metodami `area()` i `perimeter()` zwracającymi odpowiednio pole i obwód prostokąta. Dodaj metodę `is_square()` zwracającą wartość logiczną.
3. Napisz klasę `Counter` z atrybutem `value` (początkowo 0) oraz metodami `increment()`, `decrement()` i `reset()`. Utwórz dwa liczniki, wykonaj na nich różne operacje i pokaż, że ich stany są niezależne.
4. Napisz klasę `Lamp` z atrybutem `is_on` (początkowo `False`) oraz metodami `turn_on()`, `turn_off()` i `toggle()`. Dodaj metodę `status()`, która zwraca napis `"włączona"` lub `"wyłączona"`.
5. Napisz klasę `BankAccount` z atrybutami `owner` i `balance` (domyślnie 0) oraz metodami `deposit(amount)` i `withdraw(amount)`. Metody mają zgłaszać wyjątek `ValueError`, gdy kwota jest niedodatnia lub gdy na koncie brakuje środków. Przetestuj klasę, przechwytując wyjątki w bloku `try/except`.
6. Do klasy `BankAccount` z poprzedniego zadania dodaj atrybut `history` (lista) zapisujący każdą operację jako krotkę `(typ_operacji, kwota)` oraz metodę `print_history()`, która wypisuje historię operacji.
7. Do klasy `BankAccount` dodaj metodę `transfer(other, amount)`, która przelewa pieniądze na inne konto, wywołując metody `withdraw()` i `deposit()`. Utwórz dwa konta i wykonaj przelew. Sprawdź, co dzieje się z saldami, gdy przelew się nie powiedzie.
8. Przeanalizuj poniższy kod i wyjaśnij w komentarzu, dlaczego zmiana atrybutu obiektu `b` zmienia też obiekt `a`. Następnie utwórz drugi, niezależny obiekt.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
a = Point(1, 2)
b = a
b.x = 100
print(a.x)
```

9. Napisz klasę `Point` z atrybutami `x` i `y` oraz metodami `move(dx, dy)` (przesuwa punkt w miejscu) i `distance_to(other)` (zwraca odległość do innego punktu). Utwórz trzy punkty i wypisz odległości między każdą parą.
10. Napisz klasę `Student` z atrybutami `name` oraz `grades` (pusta lista tworzona w inicjalizatorze) i metodami `add_grade(grade)` (sprawdzającą, czy ocena należy do zbioru `{2, 3, 3.5, 4, 4.5, 5}`), `average()` oraz `has_passed()` (zwraca `True`, jeśli żadna ocena nie jest równa 2).
11. Utwórz listę pięciu obiektów klasy `Student` z poprzedniego zadania, dodaj im po kilka ocen, a następnie:
    - wypisz imię i średnią każdego studenta,
    - znajdź studenta z najwyższą średnią,
    - utwórz listę imion studentów, którzy nie zaliczyli przedmiotu.
12. Napisz klasę `Car` z atrybutem klasowym `wheels = 4` oraz atrybutami instancji `brand` i `model`. Wypisz atrybut `wheels` za pośrednictwem klasy i obiektu. Następnie przypisz `car.wheels = 3` dla jednego obiektu i sprawdź wartość atrybutu w innych obiektach oraz w klasie. Wyjaśnij wynik w komentarzu.
13. Napisz klasę `Ticket` z atrybutem klasowym `counter`, który zlicza liczbę utworzonych biletów. Każdy nowo utworzony bilet ma otrzymywać kolejny numer jako atrybut instancji `number`. Utwórz kilka biletów i wypisz ich numery oraz łączną liczbę biletów.
14. Napisz klasę `ShoppingCart` z metodami `add(product, price, quantity=1)`, `remove(product)`, `total()` oraz `print_receipt()`. Produkty przechowuj w słowniku, w którym kluczem jest nazwa produktu, a wartością krotka `(cena, ilość)`. Dodanie istniejącego produktu ma zwiększać jego ilość.
15. Napisz klasę `Stack` (stos) przechowującą elementy w liście, z metodami `push(item)`, `pop()`, `peek()`, `is_empty()` i `size()`. Metody `pop()` i `peek()` wywołane na pustym stosie mają zgłaszać wyjątek `IndexError` z czytelnym komunikatem. Wykorzystaj stos do sprawdzenia, czy nawiasy w napisie (np. `"(a[b]{c})"`) są poprawnie sparowane.
16. Napisz klasę `Library` przechowującą książki w postaci słownika `{tytuł: liczba_egzemplarzy}`, z metodami `add_book(title, copies=1)`, `borrow(title)` i `give_back(title)`. Metoda `borrow` ma zgłaszać wyjątek, gdy książki nie ma w bibliotece lub wszystkie egzemplarze są wypożyczone. Dodaj metody `save(path)` i `load(path)`, które zapisują i odczytują stan biblioteki z pliku tekstowego.

17 Inicjalizator

Dla każdej klasy należy poza klasą stworzyć obiekt co najmniej jeden raz.

1. Napisz klasę `Book` z inicjalizatorem przyjmującym dwa argumenty — `title` i `author`. Oba mają być zapisane jako atrybuty instancji. Utwórz dwa obiekty tej klasy i wypisz ich atrybuty.
2. Napisz klasę `Circle` z inicjalizatorem przyjmującym argument `radius`. Dodaj atrybut instancji `area`, który jest obliczany automatycznie w momencie tworzenia obiektu na podstawie przekazanego promienia.
3. Napisz klasę `Student` z inicjalizatorem przyjmującym `name` oraz opcjonalny argument `grade` z wartością domyślną "A". Utwórz jednego studenta z oceną domyślną i drugiego z oceną podaną jawnie.
4. Napisz klasę `Color` z inicjalizatorem przyjmującym trzy argumenty — `red`, `green` i `blue`, każdy z wartością domyślną 0. Utwórz obiekt, przekazując argumenty wyłącznie jako kluczowe (keyword arguments).
5. Napisz klasę `ShoppingList` z inicjalizatorem przyjmującym `owner` jako argument obowiązkowy. Inicjalizator powinien dodatkowo tworzyć pusty atrybut `items` będący listą. Nie przekazuj tej listy jako argumentu.
6. Napisz klasę `Config` z inicjalizatorem przyjmującym `app_name` jako argument obowiązkowy. Inicjalizator powinien tworzyć pusty atrybut `settings` będący słownikiem. Dodaj potem ręcznie dwie pary klucz-wartość do tego słownika na utworzonym obiekcie.
7. Napisz klasę `Playlist` z inicjalizatorem przyjmującym `name` oraz opcjonalny argument `songs`, który domyślnie ma być pustą listą. Rozwiąż poprawnie problem mutowalnego argumentu domyślnego, używając wartości None i warunkowego przypisania wewnątrz inicjalizatora.
8. Napisz klasę `TemperatureReading` z inicjalizatorem przyjmującym `value` i `unit` z wartością domyślną "Celsius". Dodaj walidację — jeśli `unit` nie jest jednym z ciągów "Celsius", "Fahrenheit" lub "Kelvin", inicjalizator powinien rzucić ValueError z odpowiednim komunikatem.
9. Napisz klasę `User` z inicjalizatorem przyjmującym `username` i `email`. Dodaj walidację sprawdzającą, czy `email` zawiera znak "@" — jeśli nie, inicjalizator powinien rzucić ValueError. Przetestuj tworzenie obiektu z poprawnym i niepoprawnym adresem.

10. Napisz klasę `BankAccount` z inicjalizatorem przyjmującym `owner` i opcjonalny `balance` z wartością domyślną 0. Dodaj walidację — jeśli `balance` jest liczbą ujemną, inicjalizator powinien rzucić ValueError. Utwórz konto z saldem dodatnim i przetestuj próbę utworzenia konta z saldem ujemnym.

11. Napisz klasę `Product` z inicjalizatorem przyjmującym `name`, `price` i opcjonalny `tags` (lista tagów z domyślną wartością None). Dodaj walidację — `price` musi być liczbą dodatnią, a `name` nie może być pustym ciągiem znaków. Rozwiąż problem mutowalnego argumentu domyślnego dla `tags`.

12. Napisz klasę `Scoreboard` z inicjalizatorem przyjmującym `game_name` oraz opcjonalny `scores` będący słownikiem (domyślnie None). Wewnątrz inicjalizatora utwórz kopię przekazanego słownika zamiast przypisywać referencję bezpośrednio. Pokaż, dlaczego to ma znaczenie, modyfikując oryginalny słownik po utworzeniu obiektu.

13. Napisz klasę `DateRange` z inicjalizatorem przyjmującym `start_date` i `end_date` jako ciągi znaków w formacie "YYYY-MM-DD". Inicjalizator powinien konwertować je na obiekty `datetime.date` i walidować, czy `start_date` nie jest późniejsza niż `end_date`.

14. Napisz klasę `Survey` z inicjalizatorem przyjmującym `title`, `*questions` (dowolną liczbę pytań jako `*args`) oraz opcjonalny `anonymous` z wartością domyślną True. Zapisz pytania jako listę w atrybucie instancji. Utwórz obiekt z pięcioma pytaniami.

15. Napisz klasę `ServerConfig` z inicjalizatorem przyjmującym `host` i `port` jako argumenty wyłącznie pozycyjne (z użyciem `/`) oraz `timeout` i `max_retries` jako argumenty wyłącznie kluczowe (z użyciem `*`), oba z wartościami domyślnymi. Pokaż poprawne i błędne wywołanie.

16. Napisz klasę `Matrix` z inicjalizatorem przyjmującym argument `data` będący listą list (macierz dwuwymiarowa). Dodaj walidację sprawdzającą, czy wszystkie wewnętrzne listy mają tę samą długość — jeśli nie, rzuć ValueError. Zapisz także atrybuty `rows` i `cols` obliczone na podstawie danych.

17. Napisz klasę `PasswordPolicy` z inicjalizatorem przyjmującym `min_length`, `require_uppercase`, `require_digit` i `require_special_char` — wszystkie z rozsądnymi wartościami domyślnymi. Dodaj walidację — `min_length` musi być liczbą całkowitą z zakresu od 6 do 128, a pozostałe argumenty muszą być typu bool. Rzuć TypeError lub ValueError w przypadku niespełnienia warunków.

18. Napisz klasę `EventLog` z inicjalizatorem przyjmującym `source` i opcjonalny `events` (lista słowników, domyślnie None). Każdy słownik w liście musi zawierać klucze "timestamp" i "message" — dodaj walidację sprawdzającą to dla każdego elementu. Inicjalizator powinien tworzyć głęboką kopię listy słowników za pomocą `copy.deepcopy`.

19. Napisz klasę `QueryBuilder` z inicjalizatorem przyjmującym `table_name` i `**default_filters`. Inicjalizator powinien walidować, czy `table_name` jest niepustym ciągiem złożonym wyłącznie z liter, cyfr i podkreślników (ochrona przed SQL injection). Wartości z

`**default_filters` mają być zapisane jako słownik w atrybucie instancji. Utwórz obiekt z trzema filtrami.

20. Napisz klasę `Pipeline` z inicjalizatorem przyjmującym `name`, `*steps` i `**options`. Każdy element `steps` musi być obiektem wywoływalnym (callable) — dodaj walidację z użyciem funkcji `callable()`. Opcje z `**options` mają być zapisane w atrybucie `config` jako słownik. Dodaj walidację sprawdzającą, czy `name` pasuje do wzorca regex `^[a-z][a-z0-9_]*$`. Utwórz obiekt, przekazując trzy funkcje lambda jako kroki i dwie opcje konfiguracyjne.

18 Właściwości

Poza klasą należy stworzyć co najmniej jeden obiekt i przetestować właściwości.

1. Stwórz klasę `Circle` z prywatnym atrybutem `__radius`. Dodaj właściwość `radius`, która zwraca wartość promienia oraz setter, który ustawia promień tylko jeśli podana wartość jest liczbą dodatnią.
2. Stwórz klasę `Person` z prywatnym atrybutem `__name`. Dodaj właściwość `name` z getterem zwracającym imię oraz setterem, który przyjmuje wyłącznie wartości typu `str`.
3. Stwórz klasę `Temperature` z prywatnym atrybutem `__celsius`. Dodaj właściwość `celsius` z getterem i setterem oraz właściwość tylko do odczytu `fahrenheit`, która zwraca temperaturę przeliczoną na stopnie Fahrenheita.
4. Stwórz klasę `Product` z prywatnymi atrybutami `__name` i `__price`. Dodaj właściwości dla obu atrybutów. Setter dla `price` powinien akceptować tylko wartości typu `int` lub `float` większe lub równe zero.
5. Stwórz klasę `Rectangle` z prywatnymi atrybutami `__width` i `__height`. Dodaj właściwości z walidacją (wartości dodatnie) oraz właściwość tylko do odczytu `area`, która zwraca pole prostokąta.
6. Stwórz klasę `Student` z prywatnym atrybutem `__grades` będącym listą. Dodaj właściwość `grades`, której getter zwraca kopię listy, a setter przyjmuje tylko listy zawierające wyłącznie liczby całkowite z zakresu 1–6.
7. Stwórz klasę `Email` z prywatnym atrybutem `__address`. Dodaj właściwość `address`, której setter waliduje, czy podany ciąg znaków zawiera dokładnie jeden znak `@` oraz co najmniej jedną kropkę po nim.
8. Stwórz klasę `BankAccount` z prywatnymi atrybutami `__owner` i `__balance`. Dodaj właściwość `balance` tylko do odczytu oraz właściwość `owner`, której setter akceptuje jedynie niepuste ciągi znaków o długości co najmniej 2 znaków.
9. Stwórz klasę `Playlist` z prywatnym atrybutem `__songs` (lista stringów). Dodaj właściwość `songs`, której getter zwraca kopię listy, a setter przyjmuje wyłącznie listy, w których każdy element jest typu `str` i nie jest pusty.
10. Stwórz klasę `Config` z prywatnym atrybutem `__settings` będącym słownikiem. Dodaj właściwość `settings`, której getter zwraca kopię słownika, a setter akceptuje tylko słowniki, w których zarówno klucze, jak i wartości są typu `str`.

11. Stwórz klasę `Password` z prywatnym atrybutem `__value`. Dodaj właściwość `value`, której getter zwraca zamaskowaną postać hasła (np. `"****"`), a setter akceptuje jedynie ciągi znaków o długości co najmniej 8, zawierające przynajmniej jedną cyfrę i jedną wielką literę.

12. Stwórz klasę `DateRange` z prywatnymi atrybutami `__start` i `__end` (obiekty `datetime.date`). Dodaj właściwości dla obu atrybutów, przy czym setter `end` powinien sprawdzać, czy data końcowa nie jest wcześniejsza niż data początkowa, a setter `start` — czy nie jest późniejsza niż data końcowa.

13. Stwórz klasę `Inventory` z prywatnym atrybutem `__items` będącym słownikiem mapującym nazwy produktów (str) na ilości (int). Dodaj właściwość `items`, której getter zwraca kopię słownika, a setter waliduje, czy wszystkie klucze są niepustymi stringami i czy wartości są nieujemnymi liczbami całkowitymi.

14. Stwórz klasę `RGBColor` z prywatnymi atrybutami `__red`, `__green`, `__blue`. Dodaj właściwość dla każdego kanału z setterem akceptującym wyłącznie liczby całkowite z zakresu 0–255 oraz właściwość tylko do odczytu `hex`, zwracającą kolor w formacie szesnastkowym (np. `"#ff00ab"`).

15. Stwórz klasę `Matrix` z prywatnym atrybutem `__data` będącym listą list (macierz 2D). Dodaj właściwość `data`, której setter sprawdza, czy podana wartość jest niepustą listą list, czy wszystkie wiersze mają tę samą długość i czy wszystkie elementy są liczbami (int lub float).

16. Stwórz klasę `UserProfile` z prywatnymi atrybutami `__username`, `__age` i `__tags` (lista stringów). Dodaj właściwości dla każdego atrybutu: `username` akceptuje tylko ciągi alfanumeryczne o długości 3–20 znaków, `age` akceptuje liczby całkowite z zakresu 0–150, a `tags` akceptuje listy unikatowych, niepustych ciągów znaków o maksymalnej długości listy 10.

17. Stwórz klasę `WeightedGrades` z prywatnym atrybutem `__grades` będącym słownikiem mapującym nazwy przedmiotów (str) na krotki `(ocena, waga)`, gdzie ocena to float z zakresu 2.0–5.0, a waga to float z zakresu 0.0–1.0. Dodaj właściwość `grades` z odpowiednią walidacją oraz właściwość tylko do odczytu `weighted_average`, zwracającą średnią ważoną.

18. Stwórz klasę `IPAddress` z prywatnym atrybutem `__address`. Dodaj właściwość `address`, której setter waliduje, czy podany ciąg znaków jest poprawnym adresem IPv4 — czyli składa się z dokładnie czterech grup liczb całkowitych z zakresu 0–255, rozdzielonych kropkami. Dodaj właściwość tylko do odczytu `is_private`, zwracającą `True`, jeśli adres należy do zakresu prywatnego (np. 192.168.x.x, 10.x.x.x).

19. Stwórz klasę `Schedule` z prywatnym atrybutem `__entries` będącym słownikiem, gdzie kluczem jest dzień tygodnia (str spośród: “Monday”–“Sunday”), a wartością lista krotek `(start_hour, end_hour)` reprezentujących bloki czasowe (int, zakres 0–23). Setter powinien walidować poprawność kluczy, formatu krotek, zakresów godzin oraz to, że bloki

czasowe w ramach jednego dnia się nie nakładają. Dodaj właściwość tylko do odczytu `total_hours`.

20. Stwórz klasę `Document` z prywatnymi atrybutami `__title` (str, 1–200 znaków), `__authors` (lista niepustych stringów, co najmniej 1 autor), `__metadata` (słownik str→str) oraz `__version` (str w formacie semantycznym, np. "1.2.3"). Dodaj właściwości z pełną walidacją dla każdego atrybutu. Setter `version` powinien dodatkowo sprawdzać, czy nowa wersja jest wyższa niż aktualna. Dodaj właściwość tylko do odczytu `summary`, zwracającą sformatowany ciąg znaków zawierający tytuł, autorów i wersję.

19 Dziedziczenie i polimorfizm

1. Stwórz klasę `Animal` z atrybutem `name` oraz metodą `speak()`, która zwraca ogólny komunikat. Następnie stwórz klasę `Dog`, która dziedziczy po `Animal` i nadpisuje metodę `speak()` tak, aby zwracała dźwięk psa.

2. Zdefiniuj klasę `Shape` z metodą `area()` zwracającą 0. Utwórz klasy `Rectangle` i `Circle`, które dziedziczą po `Shape` i implementują własne wersje `area()` z odpowiednimi obliczeniami.

3. Napisz klasę `Vehicle` z atrybutami `brand` i `max_speed` oraz metodą `describe()`. Stwórz klasę `ElectricCar` dziedziczącą po `Vehicle`, która dodaje atrybut `battery_capacity` i rozszerza metodę `describe()` o informację o baterii.

4. Utwórz klasę `Employee` z atrybutami `name` i `salary` oraz metodą `get_bonus()` zwracającą 10% pensji. Stwórz klasę `Manager` dziedziczącą po `Employee`, w której `get_bonus()` zwraca 20% pensji. Zademonstruj użycie obu klas w jednej liście i iterację po niej.

5. Zdefiniuj klasę `Logger` z metodą `log(message)`. Stwórz klasy `FileLogger` i `ConsoleLogger`, które dziedziczą po `Logger` i nadpisują `log()` tak, aby wypisywały wiadomość z odpowiednim prefiksem wskazującym miejsce zapisu. Napisz funkcję `process(logger)`, która przyjmuje dowolny obiekt typu `Logger` i wywołuje na nim `log()`.

6. Stwórz klasę `BankAccount` z metodami `deposit(amount)`, `withdraw(amount)` i `get_balance()`. Następnie utwórz klasę `SavingsAccount`, która dziedziczy po `BankAccount`, ale nadpisuje `withdraw()` tak, aby nie pozwalała na wypłatę, jeśli saldo spadłoby poniżej ustalonego minimum.

7. Napisz klasę `Notification` z metodą `send(recipient, text)`. Stwórz trzy klasy pochodne: `EmailNotification`, `SmsNotification` i `PushNotification`, z których każda nadpisuje `send()` i wypisuje komunikat odpowiedni dla danego kanału. Napisz funkcję `broadcast(notifications, recipient, text)`, która iteruje po liście powiadomień i wysyła je polimorficznie.

8. Zdefiniuj klasę `Converter` z metodą `convert(value)`, która zwraca wartość bez zmian. Stwórz klasy `CelsiusToFahrenheit` i `KilometersToMiles` dziedziczące po `Converter`, które nadpisują `convert()` właściwymi formułami. Użyj funkcji `isinstance()`, aby sprawdzić typ obiektu przed konwersją.

9. Stwórz hierarchię klas: `Media` (z atrybutem `title`), a po niej dziedziczące `Book` (z `author` i `pages`), `Movie` (z `director` i `duration`) i `Podcast` (z `host` i `episodes`). Każda klasa powinna mieć metodę `info()` zwracającą opis. Napisz funkcję, która przyjmuje listę obiektów `Media` i wypisuje informacje o każdym z nich.

10. Utwórz klasę `PaymentMethod` z metodą `pay(amount)`. Stwórz klasy `CreditCard`, `PayPal` i `BankTransfer`, które dziedziczą po `PaymentMethod`. Każda klasa powinna nadpisywać `pay()` i dodatkowo implementować metodę `validate()` sprawdzającą poprawność danych. W klasie `CreditCard` użyj `super()` do wywołania bazowej wersji `pay()` przed dodaniem własnej logiki.

11. Napisz klasę `TaskRunner` z metodami `setup()`, `execute()` i `teardown()` oraz metodą `run()`, która wywołuje te trzy metody w kolejności. Stwórz klasy `DatabaseTask` i `ApiTask`, które dziedziczą po `TaskRunner` i nadpisują `setup()`, `execute()` i `teardown()` własnymi implementacjami, ale nie zmieniają metody `run()` — to przykład wzorca Template Method.

12. Zdefiniuj klasę `User` z atrybutami `username` i `role` oraz metodą `permissions()` zwracającą pustą listę. Stwórz klasy `Admin`, `Editor` i `Viewer` dziedziczące po `User`, z których każda zwraca inną listę uprawnień. Napisz funkcję `check_access(user, required_permission)`, która sprawdza, czy dany użytkownik ma wymagane uprawnienie, korzystając z polimorfizmu.

13. Stwórz klasę abstrakcyjną `Validator` (używając modułu `abc`) z abstrakcyjną metodą `validate(data)`. Utwórz klasy `EmailValidator`, `PhoneValidator` i `AgeValidator`, które implementują `validate()` i zwracają `True` lub `False` w zależności od poprawności danych. Napisz funkcję `run_validations(validators, data)`, która uruchamia wszystkie walidatory i zbiera wyniki.

14. Zdefiniuj klasę `Plugin` z metodami `activate()` i `process(data)`. Stwórz co najmniej trzy klasy pochodne reprezentujące różne wtyczki (np. `UpperCasePlugin`, `TrimPlugin`, `ReplacePlugin`). Napisz klasę `Pipeline`, która przechowuje listę wtyczek i posiada metodę `run(data)` przepuszczającą dane przez kolejne wtyczki polimorficznie.

15. Stwórz przykład dziedziczenia wielokrotnego: klasę `Flyable` z metodą `fly()`, klasę `Swimmable` z metodą `swim()` oraz klasę `Duck`, która dziedziczy po obu. Zbadaj kolejność rozwiązywania metod (MRO) za pomocą `Duck.__mro__` i napisz krótki komentarz wyjaśniający wynik.

16. Zdefiniuj klasę `EventEmitter` z metodami `on(event_name, callback)` i `emit(event_name, *args)`. Stwórz klasę `ChatRoom` dziedziczącą po `EventEmitter`, która dodaje metody `join(user)`, `leave(user)` i `send_message(user, text)`, a każda z nich emituje odpowiednie zdarzenie. Następnie stwórz klasę `ModeratedChatRoom` dziedziczącą po `ChatRoom`, która nadpisuje `send_message()` i filtruje niedozwolone słowa przed wysłaniem.

17. Stwórz system serializacji: klasę `Serializer` z metodami `serialize(obj)` i `deserialize(data)`. Utwórz klasy `JsonSerializer`, `CsvSerializer` i `XmlSerializer`, które implementują te metody. Następnie napisz klasę `DataStore`, która w konstruktorze przyjmuje obiekt typu `Serializer` (kompozycja) i używa go do zapisywania i odczytywania danych — porównaj to podejście z dziedziczeniem i opisz w komentarzu, kiedy kompozycja jest lepsza.

18. Zdefiniuj metaklasę `SingletonMeta`, która sprawia, że każda klasa, która ją wykorzystuje, może mieć tylko jedną instancję. Stwórz klasę `DatabaseConnection` używającą tej metaklasy. Następnie stwórz klasę `PooledDatabaseConnection` dziedziczącą po `DatabaseConnection` i sprawdź, czy mechanizm singletona działa również dla klasy pochodnej. Opisz wynik w komentarzach.

20 Reprezentacje tekstowe obiektów

1. Zdefiniuj klasę `Dog` z atrybutami `name` i `breed`. Zaimplementuj metodę `__str__`, która zwróci tekst w formacie `name (breed)`.
2. Zdefiniuj klasę `Point` z atrybutami `x` i `y`. Zaimplementuj metodę `__repr__`, która zwróci tekst umożliwiający odtworzenie obiektu, np. `Point(3, 5)`.
3. Zdefiniuj klasę `Book` z atrybutami `title` i `author`. Zaimplementuj obie metody — `__str__` i `__repr__` — tak, aby zwracały różne reprezentacje tego samego obiektu.
4. Stwórz klasę `Temperature` z atrybutem `celsius`. Metoda `__str__` ma zwracać wartość z jednostką, np. `36.6°C`, a `__repr__` — wyrażenie konstruktora `Temperature(36.6)`.
5. Zdefiniuj klasę `Color` z atrybutami `r`, `g`, `b`. Zaimplementuj `__repr__` tak, aby spełniał warunek `eval(repr(obj)) == obj` (wymaga to również zdefiniowania `__eq__`).
6. Stwórz klasę `Student` z atrybutami `first_name`, `last_name` i `grade`. Zaimplementuj `__str__` zwracającą przyjazny opis ucznia, a `__repr__` — jednoznaczną reprezentację techniczną.
7. Napisz klasę `Fraction` z atrybutami `numerator` i `denominator`. Metoda `__str__` ma wyświetlać ułamek w formacie `3/4`, a `__repr__` — w formacie `Fraction(3, 4)`.
8. Zdefiniuj klasę `Product` z atrybutami `name` i `price`. Umieść kilka obiektów tej klasy w liście, a następnie wypisz tę listę za pomocą `print()`. Zaobserwuj, która metoda — `__str__` czy `__repr__` — zostaje wywołana dla elementów listy.
9. Stwórz klasę `Vector2D` z atrybutami `x` i `y`. Zaimplementuj `__repr__` i `__str__`, a następnie sprawdź, co zwraca `f"{obj}"`, `str(obj)`, `repr(obj)` oraz wypisanie obiektu w interpreterze interaktywnym.
10. Napisz klasę `LogEntry` z atrybutami `timestamp`, `level` i `message`. Metoda `__str__` ma zwracać czytelny wpis logu, a `__repr__` — pełną reprezentację z wszystkimi atrybutami przydatną do debugowania.
11. Zdefiniuj klasę `Currency` z atrybutami `amount` i `code` (np. `"PLN"`, `"USD"`). Zaimplementuj `__str__` formatującą kwotę z symbolem waluty, a `__repr__` — z pełną precyzją dziesiętną.
12. Stwórz klasę `Matrix` przechowującą dwuwymiarową listę liczb. Zaimplementuj `__str__` tak, aby wyświetlała macierz w czytelnej formie wierszowej, a `__repr__` — jako jednoliniowe wyrażenie konstruktora.

13. Zdefiniuj klasę bazową `Shape` z metodą `__repr__`, która automatycznie odczytuje nazwę klasy z `type(self).__name__`. Następnie stwórz klasy pochodne `Circle` i `Rectangle`, które dziedziczą tę implementację i dodają własne atrybuty.

14. Napisz klasę `LinkedList` z węzłami typu `Node`. Zaimplementuj `__repr__` dla klasy `LinkedList` tak, aby wyświetlała całą zawartość listy w formacie `LinkedList(1 -> 2 -> 3)`.

15. Stwórz klasę `Person` z atrybutem `secret_code`, który nie powinien pojawiać się w wynikach `__str__` (wyświetlanym użytkownikowi), ale powinien być widoczny w `__repr__` (przeznaczonym dla programisty).

16. Zdefiniuj klasę `Config` przechowującą słownik ustawień. Zaimplementuj `__repr__`, który wyświetla posortowane klucze i ich wartości, oraz `__str__`, który wyświetla jedynie liczbę kluczy i nazwy trzech pierwszych.

17. Napisz dekorator klasy `auto_repr`, który automatycznie generuje metodę `__repr__` na podstawie atrybutów przekazanych do `__init__`. Przetestuj go na co najmniej dwóch różnych klasach.

18. Stwórz hierarchię klas `Animal` -> `Mammal` -> `Cat`. Na każdym poziomie zaimplementuj `__repr__` z wywołaniem `super().__repr__()` i dołączeniem atrybutów specyficznych dla danego poziomu. Przeanalizuj wynikowy łańcuch reprezentacji.

19. Zdefiniuj klasę `LazyRepr`, w której metoda `__repr__` wykonuje kosztowną operację (np. symulowaną przez `time.sleep`). Następnie napisz zoptymalizowaną wersję z cache'owaniem wyniku `__repr__` przy użyciu techniki memoizacji i porównaj czasy wykonania.

20. Napisz metaklasę `ReprEnforcerMeta`, która przy tworzeniu klasy sprawdza, czy klasa definiuje zarówno `__str__`, jak i `__repr__`. Jeśli brakuje którejkolwiek z tych metod, metaklasa powinna podnieść wyjątek `TypeError` z odpowiednim komunikatem.

21 Emulacja kontenerów

1. Zdefiniuj klasę `MyList` opakowującą zwykłą listę. Zaimplementuj metodę `__len__`, która zwraca liczbę elementów, i przetestuj działanie wbudowanej funkcji `len()` na obiekcie tej klasy.

2. Stwórz klasę `Sentence` przechowującą tekst jako string. Zaimplementuj `__len__` tak, aby zwracała liczbę słów w zdaniu, a nie liczbę znaków.

3. Zdefiniuj klasę `ColorPalette` przechowującą listę kolorów. Zaimplementuj `__getitem__`, aby można było pobierać kolory po indeksie, np. `palette[2]`.

4. Rozszerz klasę `ColorPalette` o metodę `__setitem__`, umożliwiającą podmianę koloru pod wskazanym indeksem, np. `palette[0] = "red"`.

5. Napisz klasę `Matrix` przechowującą dane w płaskiej liście. Zaimplementuj `__getitem__` przyjmującą krotkę `(row, col)`, aby dostęp do elementów wyglądał jak `matrix[2, 3]`.

6. Stwórz klasę `DefaultDict` emulującą słownik z wartością domyślną. Zaimplementuj `__getitem__` tak, aby dla nieistniejącego klucza zwracała podaną przy inicjalizacji wartość zamiast rzucać `KeyError`.

7. Zdefiniuj klasę `Scoreboard` przechowującą wyniki graczy w słowniku. Zaimplementuj `__getitem__`, `__setitem__` i `__len__`, a następnie przetestuj na niej operacje `board["Alice"]`, `board["Bob"] = 10` oraz `len(board)`.

8. Napisz klasę `ImmutableList`, która implementuje `__getitem__` i `__len__`, ale celowo nie definiuje `__setitem__`. Sprawdź, jaki wyjątek zostanie rzucony przy próbie przypisania wartości do elementu.

9. Stwórz klasę `RangeMap`, w której `__getitem__` przyjmuje liczbę całkowitą i zwraca nazwę zakresu, do którego ta liczba należy (np. `0-10` to `"low"`, `11-20` to `"medium"`). Zakresy definiowane są przy inicjalizacji.

10. Zdefiniuj klasę `CircularBuffer` o stałym rozmiarze. Zaimplementuj `__getitem__`, `__setitem__` i `__len__` tak, aby indeksy wykraczające poza rozmiar bufora automatycznie zawijały się cyklicznie.

11. Napisz klasę `SparseVector` przechowującą jedynie niezerowe elementy w słowniku. Zaimplementuj `__getitem__` (zwracającą `0` dla brakujących indeksów), `__setitem__` (usuwającą klucz przy ustawieniu wartości `0`) oraz `__len__` (zwracającą logiczny rozmiar wektora).

12. Stwórz klasę `History` emulującą kontener z możliwością cofania zmian. Metoda `__setitem__` powinna zapisywać poprzednią wartość klucza na stosie, a dodatkowa metoda `undo(key)` powinna przywracać ostatnią wartość.

13. Zdefiniuj klasę `SliceableString`, która implementuje `__getitem__` z pełnym wsparciem dla wycinków (slice). Wyrażenie `obj[1:5]` powinno zwracać nowy obiekt `SliceableString`, a `obj[0]` — pojedynczy znak.

14. Napisz klasę `Table` z danymi w formie listy słowników. Zaimplementuj `__getitem__` tak, aby obsługiwała trzy tryby dostępu — po indeksie wiersza (int), po nazwie kolumny (string) oraz po krotce `(row_index, column_name)`.

15. Stwórz klasę `ValidatedList`, w której `__setitem__` sprawdza typ wstawianej wartości i rzuca `TypeError`, jeśli nie jest on zgodny z typem podanym przy inicjalizacji (np. `ValidatedList(int)`).

16. Zdefiniuj klasę `NestedDict` umożliwiającą dostęp do zagnieżdżonych danych za pomocą klucza w formie ścieżki z kropkami, np. `config["database.host"]` zamiast `config["database"]["host"]`. Zaimplementuj zarówno `__getitem__`, jak i `__setitem__`.

17. Napisz klasę `TimeSeries` przechowującą pary `(datetime, value)`. Zaimplementuj `__getitem__` tak, aby akceptowała zarówno indeks liczbowy, jak i obiekt `datetime` (wyszukujący najbliższy punkt w czasie), a także wycinek z obiektami `datetime` jako granicami zakresu.

18. Stwórz klasę `ChainMap` przyjmującą kilka słowników w konstruktorze. Zaimplementuj `__getitem__` przeszukującą słowniki po kolei, `__setitem__` zapisującą zawsze do pierwszego słownika, oraz `__len__` zwracającą liczbę unikalnych kluczy ze wszystkich słowników.

19. Zdefiniuj klasę `DatabaseTable` emulującą tabelę bazy danych. Zaimplementuj `__getitem__` obsługującą zapytania w stylu `table[lambda row: row["age"] > 18]`, zwracającą nowy obiekt `DatabaseTable` z przefiltrowanymi wierszami. Dodaj `__len__` i `__setitem__` do aktualizacji wierszy spełniających warunek.

20. Napisz klasę `TensorView`, która opakowuje jednowymiarową listę danych i interpretuje ją jako tensor o zadanym kształcie (np. `(2, 3, 4)`). Zaimplementuj `__getitem__` z obsługą wielowymiarowego indeksowania i wycinków na każdym wymiarze, `__setitem__` z analogiczną logiką oraz `__len__` zwracającą rozmiar pierwszego wymiaru. Upewnij się, że widok współdzieli dane z oryginałem.

22 Emulacja arytmetyki

1. Napisz klasę `Temperature` z atrybutem `celsius`. Zaimplementuj metodę `__add__`, która umożliwi dodawanie dwóch obiektów `Temperature` — wynikiem powinien być nowy obiekt z sumą wartości.

2. Napisz klasę `Counter` przechowującą liczbę całkowitą. Zaimplementuj metody `__add__` i `__sub__`, tak aby można było dodawać i odejmować obiekty `Counter` od siebie za pomocą operatorów `+` i `-`.

3. Napisz klasę `Vector2D` z atrybutami `x` i `y`. Zaimplementuj metodę `__mul__`, która pozwoli pomnożyć wektor przez skalar (liczbę), zwracając nowy wektor ze skalowanymi współrzędnymi.

4. Napisz klasę `Price` przechowującą kwotę jako `float`. Zaimplementuj metodę `__truediv__`, tak aby operator `/` dzielił cenę przez podaną liczbę i zwracał nowy obiekt `Price`.

5. Napisz klasę `Weight` z atrybutem `kg`. Zaimplementuj metody `__add__` i `__sub__` z obsługą dodawania/odejmowania zarówno innego obiektu `Weight`, jak i zwykłej liczby (skalara).

6. Napisz klasę `Wallet` z atrybutem `balance`. Zaimplementuj `__floordiv__` (operator `//`), który dzieli saldo całkowicie przez podaną liczbę, oraz `__mod__` (operator `%`), który zwraca resztę z dzielenia.

7. Napisz klasę `DataSeries` przechowującą listę wartości numerycznych. Zaimplementuj `__add__`, który realizuje dodawanie element po elemencie (elementwise) dwóch obiektów `DataSeries` o tej samej długości. Jeśli długości się nie zgadzają, zgłoś `ValueError`.

8. Napisz klasę `Score` z atrybutem `points`. Zaimplementuj `__radd__`, tak aby wyrażenie `10 + score_obj` poprawnie delegowało operację do obiektu `Score` — przetestuj, że zarówno `score + 10`, jak i `10 + score` działają.

9. Napisz klasę `Matrix1D` przechowującą jednowymiarową listę wartości. Zaimplementuj `__mul__` obsługujący dwa scenariusze: mnożenie przez skalar oraz iloczyn element po elemencie z innym obiektem `Matrix1D`. Dodaj `__rmul__` dla wyrażeń typu `3 * matrix_obj`.

10. Napisz klasę `Measurement` z atrybutami `value` i `unit`. Zaimplementuj `__add__` i `__sub__`, które pozwalają łączyć jedynie pomiary o tym samym `unit` — przy niezgodności jednostek zgłoś `TypeError` z komunikatem informującym o konflikcie.

11. Napisz klasę `Percentage` z atrybutem `value` (np. 75 oznacza 75%). Zaimplementuj `__mul__`, tak aby `Percentage(50) * 200` zwracało `float` równy 100.0 (obliczenie procentu z liczby), a mnożenie dwóch obiektów `Percentage` zwracało nowy obiekt `Percentage` ze złożonym procentem.

12. Napisz klasę `BankAccount` z atrybutem `balance`. Zaimplementuj `__iadd__` i `__isub__` (operatory `+=` i `-=`), tak aby modyfikowały saldo w miejscu. W przypadku `__isub__` zapobiegaj zejściu poniżej zera, zgłaszając `ValueError`.

13. Napisz klasę `ComplexNumber` z atrybutami `real` i `imag`. Zaimplementuj metody `__add__`, `__sub__`, `__mul__` zgodnie z regułami arytmetyki liczb zespolonych. Zaimplementuj również `__repr__`, aby wynik wyświetlał się w formacie `a + bi`.

14. Napisz klasę `Polynomial` reprezentującą wielomian za pomocą listy współczynników (indeks listy odpowiada potędze). Zaimplementuj `__add__` (dodawanie wielomianów) i `__mul__` (mnożenie wielomianów przez skalar). Zwróć uwagę na wielomiany różnych stopni.

15. Napisz klasę `TimeDelta` z atrybutem `seconds`. Zaimplementuj `__add__`, `__sub__`, `__mul__` (mnożenie przez skalar) oraz `__truediv__`. Operator `/` powinien rozróżniać dzielenie przez liczbę (zwraca `TimeDelta`) od dzielenia przez inny `TimeDelta` (zwraca `float` — iloraz czasów).

16. Napisz klasę `Vector3D` z atrybutami `x`, `y`, `z`. Zaimplementuj `__add__`, `__sub__`, `__mul__` (iloczyn skalarny z innym wektorem zwracający `float`, lub skalowanie wektora liczbą), `__rmul__` oraz `__neg__` (operator unarny `-`). Wykorzystaj type hinting i `isinstance` do rozróżniania typów operandów.

17. Napisz klasę `Currency` z atrybutami `amount` i `code` (np. "PLN", "USD"). Zaimplementuj `__add__`, `__sub__`, `__mul__` (przez skalar) i `__truediv__` (przez skalar). Operacje dodawania i odejmowania między różnymi walutami powinny być zabronione (zgłoś `TypeError`). Dodaj metody `__radd__` i `__rmul__` dla przemienności ze skalarami.

18. Napisz klasę `FractionNumber` z atrybutami `numerator` i `denominator`. Zaimplementuj `__add__`, `__sub__`, `__mul__`, `__truediv__` z pełną arytmetyką ułamków (ze sprowadzaniem do wspólnego mianownika). Każda operacja powinna zwracać ułamek w postaci skróconej — wykorzystaj `math.gcd` do upraszczania wyniku.

19. Napisz klasę `SparseVector`, która przechowuje wektor rzadki jako słownik (klucze to indeksy niezerowych elementów). Zaimplementuj `__add__`, `__sub__`, `__mul__` (iloczyn skalarny z innym `SparseVector` zwracający `float`, lub skalowanie przez liczbę). Zadbaj o poprawne łączenie kluczy i usuwanie zer z wynikowego słownika.

20. Napisz rodzinę klas reprezentujących jednostki fizyczne: `Distance` (metry), `Time` (sekundy) i `Speed` (metry na sekundę). Zaimplementuj operatory arytmetyczne tak, aby `Distance / Time` zwracało `Speed`, a `Speed * Time` zwracało `Distance`. Obsłuż również

dodawanie i odejmowanie obiektów tego samego typu oraz mnożenie przez skalar. W przypadku nieprawidłowych operacji (np. `Distance + Time`) zgłoś `TypeError`.

23 Emulacja porównania

1. Napisz klasę Temperature z atrybutem degrees. Zaimplementuj metodę `__eq__`, która porównuje dwa obiekty tej klasy na podstawie wartości degrees. Sprawdź działanie operatora `==` na dwóch instancjach.

2. W klasie Product z atrybutami name i price zaimplementuj metodę `__lt__`, tak aby operator `<` porównywał produkty po cenie. Utwórz trzy obiekty i sprawdź wynik porównania.

3. Rozszerz klasę Product z poprzedniego zadania o metodę `__le__`. Upewnij się, że metoda poprawnie obsługuje przypadek, gdy ceny są równe.

4. Napisz klasę Student z atrybutami name i grade. Zaimplementuj metody `__eq__` i `__lt__`, a następnie posortuj listę studentów za pomocą wbudowanej funkcji `sorted()`.

5. W klasie DataPoint z atrybutami label i value zaimplementuj metodę `__eq__`, która zwraca `NotImplemented`, gdy other nie jest instancją DataPoint. Wyjaśnij w komentarzu, dlaczego zwracamy `NotImplemented` zamiast `False`.

6. Napisz klasę Rectangle z atrybutami width i height. Zaimplementuj metody `__eq__`, `__lt__` i `__gt__`, tak aby porównania opierały się na polu prostokąta.

7. Użyj dekoratora `@functools.total_ordering` w klasie Score z atrybutem points. Zdefiniuj tylko `__eq__` i `__lt__`, a następnie przetestuj działanie wszystkich sześciu operatorów porównania (`==`, `!=`, `<`, `<=`, `>`, `>=`).

8. Napisz klasę SemanticVersion z atrybutami major, minor i patch. Zaimplementuj metody `__eq__` i `__lt__` porównujące wersje leksykograficznie (najpierw major, potem minor, potem patch). Posortuj listę wersji.

9. W klasie Interval z atrybutami start i end zaimplementuj metodę `__eq__` porównującą oba końce przedziału oraz `__lt__` porównującą przedziały po ich długości. Sprawdź, czy dwa przedziały o różnych końcach, ale tej samej długości, nie są sobie równe.

10. Napisz klasę Employee z atrybutami name, department i salary. Zaimplementuj `__eq__` porównujące po name i department (bez salary) oraz `__lt__` porównujące wyłącznie po salary. Zastanów się w komentarzu, jakie konsekwencje ma taka niespójna definicja równości i porządku.

11. Napisz klasę Vector2D z atrybutami x i y. Zaimplementuj `__eq__` porównujące oba składniki. Następnie zdefiniuj `__lt__` porównujące wektory po ich długości euklidesowej. Dodaj type hints do wszystkich metod.

12. W klasie Exam z atrybutami subject i score zaimplementuj `__eq__` i `__lt__`. Następnie utwórz listę obiektów Exam i wyznacz z niej element maksymalny za pomocą wbudowanej funkcji `max()` — bez podawania parametru key.

13. Napisz klasę TemperatureReading z atrybutem celsius. Zaimplementuj metody `__eq__` i `__lt__`, które porównują temperatury z zadaną tolerancją epsilon (np. 0.01). Dwie temperatury różniące się o mniej niż epsilon powinny być uznawane za równe.

14. Stwórz klasę Money z atrybutami amount i currency. Zaimplementuj `__eq__` i `__lt__`, które zgłaszają wyjątek ValueError, gdy próbujemy porównać kwoty w różnych walutach. Przetestuj oba scenariusze — zgodne i niezgodne waluty.

15. Napisz klasę Card z atrybutami rank i suit, reprezentującą kartę do gry. Zdefiniuj słownik z porządkiem rang (np. "2" do "A") i zaimplementuj `__eq__` oraz `__lt__` uwzględniające ten porządek. Użyj `@total_ordering`. Posortuj talię pięciu kart.

16. W klasie TimeSeries przechowującej listę wartości zaimplementuj `__eq__`, który porównuje serie element po elemencie, oraz `__lt__`, który porównuje serie po ich średniej arytmetycznej. Obsłuż przypadek serii o różnych długościach w `__eq__` (powinny być nierówne).

17. Napisz klasę Matrix2x2 przechowującą macierz 2x2 jako listę list. Zaimplementuj `__eq__` porównujące element po elemencie. Zdefiniuj `__lt__` porównujące macierze po ich wyznacznikach (determinant). Dodaj metodę `__repr__` wyświetlającą macierz w czytelny sposób.

18. Stwórz hierarchię klas: bazową Shape oraz pochodne Circle i Square. W klasie Shape zdefiniuj abstrakcyjne metody area(). Zaimplementuj `__eq__` i `__lt__` w klasie Shape porównujące po area(). Sprawdź, czy porównanie Circle z Square działa poprawnie dzięki dziedziczeniu.

19. Napisz klasę CacheEntry z atrybutami key, value i priority. Zaimplementuj `__eq__` porównujące po key, a `__lt__` porównujące po priority. Następnie użyj modułu heapq, aby zbudować kolejkę priorytetową opartą na obiektach CacheEntry — heapq wymaga poprawnie zdefiniowanego operatora `<`.

20. Napisz klasę DataFrame (uproszczoną) przechowującą słownik kolumn (nazwa -> lista wartości). Zaimplementuj `__eq__` sprawdzające równość struktury i danych kolumna po kolumnie. Zdefiniuj `__lt__` porównujące ramki po liczbie wierszy. Dodatkowo zaimplementuj `__hash__` tak, aby obiekty DataFrame były niehasowalne (jak w pandas) — przypisz `__hash__ = None` — i wyjaśnij w komentarzu, dlaczego zmiana `__eq__` wymaga przemyślenia `__hash__`.

Część III

Zaawansowane zagadnienia

24 Dekoratory

Poza definicją dekoratora należy go wywołać co najmniej raz.

1. Napisz dekorator `call_counter`, który zlicza, ile razy dana funkcja została wywołana. Licznik przechowuj jako atrybut funkcji opakowującej. Po każdym wywołaniu wypisz na konsolę aktualną wartość licznika.
2. Napisz dekorator `log_call`, który przed wywołaniem funkcji wypisuje jej nazwę, a po wywołaniu wypisuje zwróconą wartość. Użyj atrybutu `__name__` oryginalnej funkcji.
3. Napisz dekorator `timer`, który mierzy czas wykonania funkcji w sekundach (moduł `time`) i wypisuje go po zakończeniu. Zwróć wynik oryginalnej funkcji bez zmian.
4. Napisz dekorator `suppress_errors`, który przechwytuje wszystkie wyjątki rzucane przez funkcję, wypisuje komunikat z treścią błędu i zwraca `None` zamiast propagować wyjątek.
5. Napisz dekorator `repeat(n)`, który przyjmuje argument `n` i powoduje, że dekorowana funkcja zostaje wywołana `n` razy z tymi samymi argumentami. Dekorator zwraca wynik ostatniego wywołania.
6. Napisz dekorator `cache_result`, który zapamiętuje wyniki wywołań funkcji w słowniku, używając argumentów pozycyjnych jako klucza. Jeśli funkcja została już wywołana z tymi samymi argumentami, zwróć wynik ze słownika bez ponownego wywołania.
7. Napisz dekorator `validate_types`, który przyjmuje typy jako argumenty pozycyjne (np. `@validate_types(int, int)`). Przed wywołaniem funkcji sprawdź, czy przekazane argumenty mają odpowiednie typy. Jeśli nie, rzuć `TypeError` z opisowym komunikatem.
8. Napisz dekorator `retry(max_attempts, delay)`, który w przypadku wyjątku ponawia wywołanie funkcji do `max_attempts` razy, odczekując `delay` sekund między próbami. Jeśli wszystkie próby się nie powiodą, rzuć ostatni wyjątek.
9. Napisz dekorator `flatten_result`, który po wywołaniu funkcji sprawdza, czy wynik jest zagnieżdżoną listą (listą list). Jeśli tak, spłaszcza ją do jednopoziomowej listy. Jeśli wynik nie jest listą, zwraca go bez zmian.
10. Napisz dekorator `sort_result(key=None, reverse=False)`, który sortuje wynik funkcji, jeśli jest on listą. Parametry `key` i `reverse` działają tak samo jak w funkcji wbudowanej `sorted`. Jeśli wynik nie jest listą, zwróć go bez zmian.

11. Napisz dekorator `transform_keys(func)`, który przyjmuje funkcję transformującą (np. `str.upper`). Jeśli dekorowana funkcja zwraca słownik, zastosuj transformację do wszystkich kluczy tego słownika. Jeśli wynik nie jest słownikiem, zwróć go bez zmian.

12. Napisz dekorator `ensure_keys(*required_keys)`, który sprawdza, czy słownik zwrócony przez dekorowaną funkcję zawiera wszystkie wymagane klucze. Jeśli brakuje któregoś klucza, rzuć `KeyError` z listą brakujących kluczy. Jeśli wynik nie jest słownikiem, rzuć `TypeError`.

13. Napisz dekorator `paginate(page_size)`, który przyjmuje rozmiar strony. Jeśli dekorowana funkcja zwraca listę, dekorator zamienia ją na listę podlist, z których każda ma co najwyżej `page_size` elementów. Jeśli wynik nie jest listą, zwróć go bez zmian.

14. Napisz dekorator `filter_result(predicate)`, który przyjmuje funkcję filtrującą. Jeśli dekorowana funkcja zwraca listę, dekorator pozostawia tylko te elementy, dla których `predicate` zwraca `True`. Jeśli wynik nie jest listą, zwróć go bez zmian.

15. Napisz dekorator `default_args(**defaults)`, który przyjmuje domyślne wartości argumentów nazwanych. Jeśli dekorowana funkcja zostanie wywołana bez któregoś z tych argumentów, dekorator automatycznie go doda. Użyj modułu `inspect` do zbadania sygnatury funkcji.

16. Napisz dekorator `audit_log(log_list)`, który przyjmuje zewnętrzną listę jako argument. Po każdym wywołaniu dekorowanej funkcji dopisuje do tej listy słownik zawierający klucze: `function_name`, `args`, `kwargs`, `result` oraz `timestamp` (użyj `datetime.datetime.now()`).

17. Napisz dekorator `compose(*decorators)`, który przyjmuje dowolną liczbę innych dekoratorów i stosuje je kolejno do dekorowanej funkcji (pierwszy dekorator na liście opakowuje funkcję jako pierwszy, ostatni jako ostatni). Przetestuj go, łącząc co najmniej dwa wcześniej napisane dekoratory.

18. Napisz dekorator `rate_limit(max_calls, period)`, który pozwala na co najwyżej `max_calls` wywołań dekorowanej funkcji w ciągu `period` sekund. Jeśli limit zostanie przekroczony, rzuć wyjątek `RuntimeError` z informacją, ile sekund zostało do odblokowania.

19. Napisz dekorator `dispatch_on_type`, który rejestruje różne implementacje funkcji w zależności od typu pierwszego argumentu. Dekorator udostępnia metodę `register(type, func)` na funkcji opakowującej. Przy wywołaniu sprawdza typ pierwszego argumentu i deleguje do odpowiedniej zarejestrowanej implementacji. Jeśli typ nie jest zarejestrowany, wywołuje domyślną funkcję.

20. Napisz dekorator `pipeline(*transformers)`, który przyjmuje dowolną liczbę funkcji transformujących. Po wywołaniu dekorowanej funkcji przepuszcza jej wynik kolejno przez każdy transformer. Jeśli którykolwiek transformer rzuci wyjątek, przerwij łańcuch, wypisz

komunikat diagnostyczny ze wskazaniem, który transformer zawiódł, i zwróć wynik sprzed tego kroku.

25 Wyrażenia regularne

1. Zaimportuj moduł `re` i za pomocą `re.findall()` znajdź w tekście `"Cena: 49.99 PLN, rabat: 10.50 PLN"` wszystkie liczby zmiennoprzecinkowe, używając wzorca `r"\d+\.\d+"`.

2. Wyjaśnij, jaka jest różnica między zapisem wzorca `"\\d+\\.\\d+"` a `r"\d+\.\d+"`. Który z nich jest czytelniejszy i dlaczego?

3. Dany jest string `text = "Raport_2024 raport_2025 RAPORT_2026"`. Użyj `re.findall()` z wzorcem zawierającym `\d{4}`, aby wyodrębnić z niego wszystkie czterocyfrowe liczby.

4. Dany jest string `log = "ERROR: file not found at /data/input.csv"`. Napisz wzorzec z `r"\b...\b"`, który wyszuka w nim dokładnie słowo `file` jako całe słowo (nie jako fragment innego wyrazu).

5. Mając listę `files = ["data.csv", "report.pdf", "sales.xlsx", "model.pkl"]`, napisz wyrażenie z `re.search()` i wzorcem wykorzystującym alternację `(csv|xlsx)` oraz kotwicę `$`, które odfiltruje tylko pliki z rozszerzeniem csv lub xlsx.

6. Dany jest string `record = "  Jan Kowalski  (ur. 1990) "`. Użyj `re.sub()` z odpowiednim wzorcem w r-stringu, aby usunąć z niego fragment w nawiasach zawierający rok urodzenia.

7. Mając string `messy = "Anna   NOWAK   (ur.  1985)"`, zastosuj `re.sub(r"\s+", " ", messy)` do zamiany wielokrotnych białych znaków na pojedynczą spację. Opisz, co robi tu kwantyfikator `+`.

8. Napisz wyrażenie regularne w r-stringu, które dopasuje opcjonalną kropkę po skrócie `ur` — wykorzystaj kwantyfikator `?` we wzorcu `r"ur\.?"`.

9. Zdefiniuj zmienną `column = "price"` i za pomocą rf-stringa utwórz wzorzec `rf"\b{column}\b"`. Następnie użyj `re.findall()` na tekście `"The total_price and price are different"` i wyjaśnij, dlaczego wynik zawiera tylko `"price"`.

10. Mając listę rozszerzeń `extensions = ["csv", "xlsx", "parquet"]`, zbuduj dynamicznie wzorzec regex za pomocą rf-stringa i metody `join()`, tak aby powstał wzorzec `r"\.(csv|xlsx|parquet)$"`.

11. Dany jest string `data = "Wzrost: 180cm, waga: 75kg, wiek: 30lat"`. Napisz wzorzec z `re.findall()`, który wyodrębni wszystkie liczby (ciągi cyfr), korzystając z `r"\d+"`.

12. Mając listę rekordów `raw_data = ["Jan Kowalski (ur. 1990)", "Anna Nowak (ur. 1985)"]`, napisz pętlę `for`, w której za pomocą `re.sub()` usuniesz fragment z rokiem urodzenia, a potem usuniesz nadmiarowe spacje — tak jak w przykładzie z wykładu.

13. Napisz wzorzec w r-stringu, który dopasuje dowolny ciąg białych znaków (`\s+`) otoczony granicami słowa (`\b`). Przetestuj go na stringu `"Hello   World"` za pomocą `re.sub()`, zamieniając wielokrotne spacje na myślnik.

14. Dany jest string `line = "error: timeout at 12:45:30"`. Użyj `re.search()` z wzorcem `r"\d{2}:\d{2}:\d{2}"`, aby znaleźć w nim znacznik czasu. Wyjaśnij rolę kwantyfikatora `{2}` we wzorcu.

15. Mając listę plików `all_files = ["notes.txt", "data.csv", "image.png", "report.xlsx", "archive.parquet"]`, napisz list comprehension z `re.search()` i dynamicznie zbudowanym rf-wzorcem (na podstawie listy rozszerzeń), który zwróci tylko pliki danych.

16. Dany jest tekst `info = "PIOTR  Wiśniewski(ur.2000)"`. Zastosuj na nim kolejno dwie operacje `re.sub()`: pierwszą do usunięcia fragmentu z rokiem urodzenia (zwróć uwagę na brak spacji przed nawiasem), drugą do normalizacji spacji. Na końcu wywołaj `.strip().title()`.

17. Napisz wzorzec w r-stringu, który dopasuje adresy e-mail w uproszczonej postaci: ciąg znaków alfanumerycznych, znak `@`, ciąg znaków alfanumerycznych, kropka i domena. Wykorzystaj `\w+`, literalną kropkę `\.` oraz `re.findall()`.

18. Zdefiniuj listę zabronionych słów `banned = ["error", "fail", "crash"]`. Za pomocą rf-stringa i metody `join()` utwórz wzorzec z alternacją i granicami słowa `\b`, a następnie użyj `re.findall()` do wyszukania tych słów w przykładowym logu.

19. Dany jest wieloliniowy string z danymi w formacie `"klucz: wartość"`. Napisz wzorzec z `re.findall()` wykorzystujący grupę `()` i `\s+`, który wyodrębni same wartości (po dwukropku i spacji). Użyj r-stringa.

20. Napisz funkcję `clean_records(records)`, która przyjmuje listę stringów w formacie `"IMIĘ NAZWISKO (ur. RRRR)"`. Funkcja ma za pomocą `re.sub()` z r-stringiem usunąć rok urodzenia w nawiasie, znormalizować spacje (`\s+`), przyciąć string (`.strip()`) i zastosować `.title()`. Zwróć listę oczyszczonych rekordów. Wykorzystaj rf-string, aby wzorzec obsługiwał opcjonalną spację i opcjonalną kropkę po `ur`.

26 Iteratory i generatory

1. Napisz klasę `CountdownIterator`, która przyjmuje liczbę całkowitą `start` i przy iteracji zwraca kolejne liczby od `start` do 0 włącznie. Wykorzystaj protokół iteratora (`__iter__`, `__next__`).
2. Stwórz wyrażenie generatorowe `cubes`, które dla `range(20)` produkuje sześciany kolejnych liczb. Następnie użyj `sum()` do obliczenia ich sumy.
3. Zaimplementuj klasę `RangeIterator` z parametrami `start`, `stop` i `step`, działającą analogicznie do wbudowanego `range`, ale w pełni wykorzystującą protokół iteratora.
4. Napisz funkcję generatorową `even_numbers(limit)`, która zwraca kolejne liczby parzyste mniejsze od `limit`. Użyj jej w pętli `for` do wypisania wartości.
5. Korzystając z listy składanej, przekształć listę `raw_data = ["12", "abc", "7.5", "", "3", "xyz"]` w listę liczb całkowitych zawierającą tylko poprawne wartości całkowite.
6. Napisz klasę `FibonacciIterator`, która przy każdym wywołaniu `__next__()` zwraca kolejną liczbę z ciągu Fibonacciego. Iterator ma się zatrzymać po wygenerowaniu `n` elementów (`StopIteration`).
7. Stwórz funkcję generatorową `chunked(data, chunk_size)`, która działa podobnie do klasy `DataBatchIterator` z wykładu, ale używa instrukcji `yield` zamiast klasy.
8. Napisz słownik składany `word_lengths`, który dla listy `words = ["python", "data", "iterator", "yield", "set"]` zwróci mapowanie słowo → jego długość, ale tylko dla słów dłuższych niż 3 znaki.
9. Zaimplementuj generator `running_average(numbers)`, który dla listy liczb zwraca kolejne wartości średniej kroczącej (po pierwszym, drugim, trzecim… elemencie).
10. Napisz klasę `ReverseIterator`, która przyjmuje sekwencję i przy iteracji zwraca jej elementy w odwróconej kolejności. Nie korzystaj z funkcji `reversed()`.
11. Stwórz potok generatorów dla listy stringów `lines = ["10", "abc", "20", "", "30"]`: pierwszy generator filtruje wartości będące liczbami, drugi konwertuje je na `int`, trzeci podnosi do kwadratu. Oblicz sumę końcową używając `sum()`.
12. Napisz funkcję generatorową `cycle_n_times(iterable, n)`, która `n` razy iteruje po podanym iterowalnym obiekcie, zwracając kolejne elementy.

13. Korzystając z wyrażenia generatorowego oraz funkcji `any()`, sprawdź czy w liście `numbers = [3, 7, 11, 14, 19, 25]` znajduje się jakaś liczba podzielna przez 5. Następnie z użyciem `all()` sprawdź, czy wszystkie są mniejsze niż 30.

14. Zaimplementuj klasę `BatchIterator` z type hinting (wykorzystaj `Iterator`, `List`, `Any` z modułu `typing`), która zwraca dane w partiach o podanym rozmiarze, ale dodatkowo pomija partie zawierające mniej niż `min_size` elementów.

15. Stwórz zbiór składany `unique_first_letters` zawierający pierwsze litery (małe) słów z listy `words = ["Apple", "banana", "apricot", "Cherry", "blueberry"]`, ale wyłącznie dla słów o długości co najmniej 5 znaków.

16. Napisz funkcję generatorową `read_lines_filtered(filepath, keyword)`, która czyta plik linia po linii i zwraca tylko te linie, które zawierają podane słowo kluczowe (po `strip()`). Wykorzystaj instrukcję `with open(...)`.

17. Zaimplementuj klasę `ZipIterator`, która przyjmuje dwie sekwencje i przy iteracji zwraca krotki par odpowiadających sobie elementów. Iteracja kończy się, gdy krótsza z sekwencji się wyczerpie. Nie korzystaj z wbudowanego `zip()`.

18. Napisz potok generatorów do przetwarzania pliku CSV (analogicznie do przykładu z wykładu): `load_records` wczytuje wiersze, `filter_valid` odrzuca rekordy z brakującą wartością w kolumnie 1, `extract_feature` zwraca wartości numeryczne z wybranej kolumny. Następnie napisz funkcję `compute_stats(pipeline)`, która zwraca słownik z kluczami `min`, `max`, `mean`. Użyj type hinting (`Generator`, `List`, `Dict`).

19. Korzystając z listy składanej zagnieżdżonej, wygeneruj macierz mnożenia o wymiarach 5×5 jako listę list. Następnie z użyciem wyrażenia generatorowego oblicz sumę wszystkich elementów na przekątnej.

20. Zaimplementuj klasę `MovingWindowIterator` z type hinting, która przyjmuje sekwencję `data: List[Any]` oraz `window_size: int`. Przy iteracji zwraca kolejne podlisty (okna) o podanym rozmiarze, przesuwane o jeden element. Po wyczerpaniu możliwych okien zgłasza `StopIteration`. Dodatkowo napisz wyrażenie generatorowe wykorzystujące tę klasę do obliczenia średniej w każdym oknie dla listy liczb.

27 Operatory bitowe

1. Napisz funkcję `is_even`, która przyjmuje liczbę całkowitą i zwraca wartość logiczną. Funkcja używając operatora `&` ma sprawdzić, czy liczba jest parzysta. Funkcja ma działać poprawnie również dla liczb ujemnych.
2. Napisz funkcję `last_bit`, która przyjmuje liczbę całkowitą i zwraca wartość najmłodszego bitu (0 lub 1) tej liczby.
3. Napisz funkcję `multiply_by_power_of_two`, która przyjmuje dwie liczby całkowite `x` i `n`, a zwraca wynik mnożenia `x` przez `2^n` z wykorzystaniem przesunięcia bitowego. Następnie napisz analogiczną funkcję `divide_by_power_of_two` realizującą dzielenie całkowite.
4. Napisz funkcję `mod_power_of_two`, która przyjmuje dwie liczby całkowite `x` i `n`, a zwraca resztę z dzielenia `x` przez `2^n`, używając wyłącznie operatorów bitowych (bez `%`).
5. Napisz funkcję `get_bit`, która przyjmuje liczbę całkowitą `x` oraz pozycję `i` (liczba całkowita), a zwraca wartość bitu na pozycji `i` w liczbie `x` (licząc od pozycji 0 jako najmłodszej).
6. Napisz trzy funkcje: `set_bit`, `clear_bit` oraz `toggle_bit`. Każda z nich przyjmuje liczbę całkowitą `x` oraz pozycję `i` (liczba całkowita) i zwraca liczbę całkowitą — kolejno z ustawionym, wyzerowanym i przełączonym bitem na pozycji `i`.
7. Napisz funkcję `to_u2`, która przyjmuje liczbę całkowitą `x` oraz liczbę bitów `bits` (liczba całkowita), a zwraca reprezentację U2 liczby `x` na zadanej szerokości jako napis złożony z zer i jedynek.
8. Napisz funkcję `count_ones`, która przyjmuje nieujemną liczbę całkowitą i zwraca liczbę całkowitą równą liczbie bitów ustawionych na 1. Wykorzystaj funkcję `bin` i metodę `count`.
9. Napisz funkcję `hamming_distance`, która przyjmuje dwie liczby całkowite i zwraca liczbę całkowitą równą odległości Hamminga między nimi (liczba pozycji, na których różnią się ich bity).
10. Napisz funkcję `is_power_of_two`, która przyjmuje liczbę całkowitą i zwraca wartość logiczną informującą, czy ta liczba jest dodatnią potęgą dwójki. Użyj wzorca `x & (x - 1)`.
11. Napisz funkcję `lowest_set_bit`, która przyjmuje liczbę całkowitą `x` i zwraca liczbę całkowitą będącą wartością najmłodszego ustawionego bitu (np. dla wejścia `44` zwróci `4`). Użyj wyrażenia z `-x`.

12. Napisz funkcję `find_unique`, która przyjmuje listę liczb całkowitych i zwraca liczbę całkowitą — jedyną wartość pojawiającą się nieparzystą liczbę razy, gdy wszystkie pozostałe występują parami. Wykorzystaj XOR oraz pętlę `for`.

13. Napisz funkcję `xor_swap`, która przyjmuje dwie liczby całkowite `a` i `b`, a zwraca krotkę dwóch liczb całkowitych z zamienionymi wartościami. Nie używaj zmiennej pomocniczej — korzystaj wyłącznie z operatora `^`.

14. Napisz funkcję `reverse_bits`, która przyjmuje liczbę całkowitą `x` oraz liczbę bitów `bits` (liczba całkowita), a zwraca liczbę całkowitą powstałą przez odwrócenie kolejności bitów w `bits`-bitowej reprezentacji liczby `x` (np. dla `x = 0b1011` i `bits = 4` wynikiem jest `0b1101`).

15. Napisz funkcję `extract_byte`, która przyjmuje liczbę całkowitą `x` oraz numer bajtu `n` (liczba całkowita), a zwraca liczbę całkowitą będącą wartością `n`-tego bajtu liczby `x` (bajt zerowy to najmłodsze 8 bitów). Wykorzystaj kombinację `>>` i maski `& 0xFF`.

16. Zdefiniuj zestaw flag uprawnień korzystając z `IntFlag` i `auto`: `READ`, `WRITE`, `EXECUTE`, `ADMIN`. Napisz trzy funkcje: `grant` przyjmującą bieżące uprawnienia oraz nową flagę i zwracającą nowe uprawnienia, `revoke` przyjmującą bieżące uprawnienia oraz flagę do usunięcia i zwracającą nowe uprawnienia, `has_all` przyjmującą bieżące uprawnienia oraz wymagane flagi i zwracającą wartość logiczną informującą, czy bieżące uprawnienia zawierają wszystkie wymagane flagi.

17. Napisz klasę `Bitmask` z polami `value` (liczba całkowita) i `width` (liczba całkowita) oraz metodami: `set` (przyjmuje pozycję bitu, nic nie zwraca), `clear` (przyjmuje pozycję bitu, nic nie zwraca), `flip` (przyjmuje pozycję bitu, nic nie zwraca), `is_set` (przyjmuje pozycję bitu, zwraca wartość logiczną) oraz `__repr__` zwracającą reprezentację binarną o szerokości `width` uzupełnioną wiodącymi zerami.

18. Rozszerz klasę `BitSet` z wykładu o metodę `difference`, która przyjmuje inny obiekt `BitSet` i zwraca nowy `BitSet` zawierający elementy obecne w `self`, ale nieobecne w `other`. Dodaj również metodę `is_subset`, która przyjmuje inny obiekt `BitSet` i zwraca wartość logiczną informującą, czy `self` jest podzbiorem `other`.

19. Napisz klasę `BloomFilter` z metodami: `__init__` przyjmującą rozmiar (liczba całkowita) i nic niezwracającą, `add` przyjmującą wartość (liczba całkowita) i nic niezwracającą oraz `__contains__` przyjmującą wartość (liczba całkowita) i zwracającą wartość logiczną. Filtr ma przechowywać dane w jednym polu `bits` (liczba całkowita) i używać trzech prostych funkcji haszujących postaci `(value * k) & (size - 1)` dla `k = 1, 3, 7`. Metoda `add` ustawia odpowiednie bity, a operator `in` sprawdza, czy wszystkie te bity są ustawione. Zakładamy, że rozmiar jest potęgą dwójki.

20. Napisz klasę `Xorshift32` implementującą generator liczb pseudolosowych z rodziny xorshift. W metodzie `__init__` (przyjmuje ziarno będące liczbą całkowitą, nic nie zwraca) zapisz ziarno w polu `state` (liczba całkowita). Metoda `next` (nie przyjmuje argumentów, zwraca liczbę całkowitą) ma wykonać kolejno: `state ^= (state << 13) & 0xFFFFFFFF`,

`state ^= (state >> 17)`, `state ^= (state << 5) & 0xFFFFFFFF`, a następnie zwrócić nową wartość pola `state` zamaskowaną do 32 bitów. Dodaj metodę `random_below`, która przyjmuje liczbę całkowitą `n` i zwraca liczbę całkowitą z zakresu `[0, n)` przy założeniu, że `n` jest potęgą dwójki.

28 Wybrane typy sekwencyjne

1. Napisz funkcję `to_bytes_literal`, która przyjmuje napis ASCII i zwraca odpowiadający mu obiekt `bytes` zapisany jako literał z prefiksem `b`. Następnie wypisz typ wyniku oraz jego długość.

2. Napisz funkcję `count_byte`, która przyjmuje obiekt `bytes` oraz pojedynczy bajt (liczba całkowita z zakresu 0–255) i zwraca liczbę całkowitą równą liczbie wystąpień tego bajtu w sekwencji.

3. Napisz funkcję `pack_three_doubles`, która przyjmuje trzy liczby zmiennoprzecinkowe i zwraca obiekt `bytes` powstały przez ich serializację za pomocą `struct.pack` z formatem `"3d"`. Sprawdź długość wyniku i upewnij się, że wynosi 24.

4. Napisz funkcję `unpack_three_doubles`, która przyjmuje obiekt `bytes` o długości 24 i zwraca krotkę trzech liczb zmiennoprzecinkowych odzyskanych za pomocą `struct.unpack`.

5. Napisz funkcję `pack_vector`, która przyjmuje listę liczb zmiennoprzecinkowych o dowolnej długości i zwraca obiekt `bytes` z ich serializacją w formacie `f"{len(values)}d"`. Następnie napisz funkcję `unpack_vector` przyjmującą obiekt `bytes` i liczbę elementów (liczba całkowita), zwracającą krotkę liczb zmiennoprzecinkowych.

6. Napisz funkcję `to_mutable`, która przyjmuje obiekt `bytes` i zwraca odpowiadający mu obiekt `bytearray`. Napisz też funkcję `to_immutable`, która wykonuje konwersję w drugą stronę.

7. Napisz funkcję `replace_first_byte`, która przyjmuje obiekt `bytearray` oraz nową wartość bajtu (liczba całkowita z zakresu 0–255), zmienia pierwszy bajt na podaną wartość i nic nie zwraca. Funkcja ma modyfikować bufor w miejscu.

8. Napisz funkcję `overwrite_double`, która przyjmuje obiekt `bytearray` zawierający serializowany wektor liczb zmiennoprzecinkowych, indeks elementu (liczba całkowita) oraz nową wartość (liczba zmiennoprzecinkowa) i nadpisuje wskazany element przy użyciu `struct.pack_into` z formatem `"d"`. Funkcja nic nie zwraca.

9. Napisz funkcję `build_int_array`, która przyjmuje listę liczb całkowitych i zwraca obiekt `array.array` z kodem typu `'i'` (int32) zawierający te wartości.

10. Napisz funkcję `array_memory_usage`, która przyjmuje obiekt `array.array` i zwraca liczbę całkowitą będącą szacunkowym rozmiarem zajmowanej pamięci w bajtach. Wykorzystaj `buffer_info` oraz pole `itemsize`.

11. Napisz funkcję `compare_memory`, która przyjmuje liczbę całkowitą `n`, tworzy zwykłą listę `list(range(n))` oraz tablicę `array.array` z kodem typu `'i'` zawierającą te same elementy, a następnie wypisuje rozmiar pamięci obu struktur. Do listy użyj `sys.getsizeof`, do tablicy iloczynu długości i `itemsize`.

12. Napisz funkcję `save_floats`, która przyjmuje obiekt `array.array` z kodem typu `'d'` oraz ścieżkę do pliku (napis), a następnie zapisuje zawartość tablicy do pliku binarnego za pomocą `tofile`. Funkcja nic nie zwraca.

13. Napisz funkcję `load_floats`, która przyjmuje ścieżkę do pliku (napis) oraz liczbę elementów do odczytu (liczba całkowita) i zwraca obiekt `array.array` z kodem typu `'d'` wczytany z pliku binarnego za pomocą `fromfile`.

14. Napisz funkcję `roundtrip_array`, która przyjmuje listę liczb zmiennoprzecinkowych oraz ścieżkę do pliku (napis). Funkcja ma utworzyć tablicę `array.array` z kodem typu `'d'`, zapisać ją do pliku, odczytać z pliku i zwrócić odczytaną tablicę.

15. Napisz funkcję `array_from_generator`, która przyjmuje liczbę całkowitą `n` i zwraca obiekt `array.array` z kodem typu `'d'` zawierający `n` wartości postaci `x * 0.1` dla `x` z zakresu od 0 do `n-1`. Wykorzystaj wyrażenie generujące przekazane do konstruktora `array.array`.

16. Napisz funkcję `read_pgm_header`, która przyjmuje ścieżkę do pliku PGM (napis) i zwraca krotkę zawierającą szerokość, wysokość oraz maksymalną wartość piksela (trzy liczby całkowite). Funkcja ma otwierać plik w trybie `"rb"`, czytać linię z magicznym numerem `b"P5"`, pomijać linie komentarzy zaczynające się od `b"#"` i nie odczytywać samych pikseli.

17. Napisz funkcję `read_pgm_pixels`, która przyjmuje ścieżkę do pliku PGM (napis) i zwraca krotkę: szerokość (liczba całkowita), wysokość (liczba całkowita), maksymalna wartość (liczba całkowita) oraz obiekt `bytearray` z surowymi pikselami. Funkcja ma sprawdzać poprawność magicznego numeru `b"P5"` przy użyciu instrukcji `assert`.

18. Napisz funkcję `pixels_to_matrix`, która przyjmuje obiekt `bytearray` z pikselami obrazu, szerokość (liczba całkowita) i wysokość (liczba całkowita), a zwraca listę list liczb całkowitych reprezentującą obraz wiersz po wierszu. Wykorzystaj wycinanie sekwencji oraz konstruktor `list`.

19. Napisz funkcję `invert_pgm`, która przyjmuje obiekt `bytearray` z pikselami obrazu PGM oraz maksymalną wartość piksela (liczba całkowita), modyfikuje bufor w miejscu tak, aby każdy piksel został zastąpiony wartością `max_val - piksel` (negatyw obrazu), i nic nie zwraca. Wykorzystaj pętlę `for` po indeksach.

20. Napisz funkcję `pgm_pipeline`, która przyjmuje ścieżkę do pliku wejściowego PGM (napis) oraz ścieżkę do pliku wyjściowego (napis). Funkcja ma odczytać nagłówek i piksele obrazu (analogicznie do `read_pgm_pixels`), zapisać piksele do tablicy `array.array` z kodem typu `'b'` lub `'B'`, zapisać tę tablicę do pliku binarnego za pomocą `tofile`, a następnie odczytać ją z powrotem za pomocą `fromfile` i zwrócić odczytaną tablicę.

29 Wyrażenia lambda

1. Napisz wyrażenie lambda o nazwie `cube`, które przyjmuje liczbę całkowitą i zwraca jej sześcian. Wywołaj je dla wartości 4 i wypisz wynik.
2. Zdefiniuj wyrażenie lambda `area_rectangle` przyjmujące dwa argumenty (długość i szerokość) i zwracające pole prostokąta. Sprawdź jego działanie dla kilku par wartości.
3. Przy pomocy funkcji `map` oraz wyrażenia lambda przekształć listę liczb całkowitych `numbers` na listę ich wartości bezwzględnych. Wynik wypisz jako listę.
4. Wykorzystując `filter` oraz wyrażenie lambda, wybierz z listy `words` tylko te ciągi znaków, które mają długość większą niż 5. Wynikiem ma być nowa lista.
5. Posortuj listę krotek `points` postaci `(x, y)` rosnąco według drugiej współrzędnej, korzystając z funkcji `sorted` i parametru `key` w postaci wyrażenia lambda.
6. Mając listę słowników `students`, w której każdy element zawiera klucze `name` oraz `grade`, posortuj ją malejąco po wartości `grade` przy pomocy `sorted` i lambdy.
7. Wykorzystując `functools.reduce` oraz wyrażenie lambda, oblicz sumę kwadratów elementów listy `values` zawierającej liczby zmiennoprzecinkowe.
8. Za pomocą `reduce` i lambdy znajdź największy element w liście liczb całkowitych `numbers`, nie używając wbudowanej funkcji `max`.
9. Zdefiniuj wyrażenie lambda `classify_temperature`, które dla podanej temperatury (typu zmiennoprzecinkowego) zwraca jeden z napisów: `"cold"`, `"warm"` lub `"hot"`, w zależności od przedziału (poniżej 10, między 10 a 25, powyżej 25). Wykorzystaj wyrażenie warunkowe.
10. Dla listy ciągów znaków `names` użyj kombinacji `map` i lambdy, aby zamienić każde imię na format, w którym pierwsza litera jest wielka, a pozostałe małe. Następnie z tak przekształconej listy odfiltruj imiona zaczynające się na literę `A`.
11. Mając dwie listy o jednakowej długości — `prices` i `quantities` — wykorzystaj `zip`, `map` i lambdę, aby obliczyć listę wartości iloczynów odpowiadających sobie elementów.
12. Korzystając z `enumerate` oraz wyrażenia lambda przekazanego do `map`, utwórz listę ciągów znaków postaci `"i: value"` dla każdej pozycji listy `items`.

13. Napisz wyrażenie generatorowe `even_squares`, które dla kolejnych liczb z zakresu od 1 do 100 zwraca kwadraty liczb parzystych. Następnie wypisz pierwsze pięć wartości za pomocą `next`.

14. Korzystając z funkcji `iter` oraz `next`, pobierz ręcznie trzy pierwsze elementy z listy `letters` zawierającej znaki alfabetu. Wytłumacz w komentarzu różnicę między obiektem iterowalnym a iteratorem.

15. Zdefiniuj funkcję `make_multiplier`, która przyjmuje liczbę `factor` i zwraca wyrażenie lambda mnożące swój argument przez tę liczbę. Pokaż działanie tworząc funkcje `double` oraz `triple`.

16. Mając listę słów `text_tokens`, użyj `sorted` wraz z lambdą, aby uporządkować je najpierw rosnąco po długości słowa, a następnie alfabetycznie (wykorzystaj krotkę jako wartość zwracaną przez klucz).

17. Korzystając wyłącznie z `map`, `filter` i lambd, napisz wyrażenie obliczające sumę sześcianów liczb nieparzystych z zakresu od 1 do 50. Do końcowego sumowania wykorzystaj funkcję `sum`.

18. Zaimplementuj funkcję generatorową `running_average`, która dla podanego obiektu iterowalnego z liczbami zwraca kolejne średnie kroczące. Następnie, używając `map` i lambdy, zaokrąglij wyniki do dwóch miejsc po przecinku.

19. Mając listę słowników `transactions` z kluczami `category` oraz `amount`, oblicz przy pomocy `reduce` i lambdy słownik agregujący sumy kwot w obrębie każdej kategorii. Akumulatorem ma być pusty słownik.

20. Napisz funkcję `compose`, która przyjmuje dowolną liczbę funkcji jednoargumentowych (przekazywanych jako wyrażenia lambda) i zwraca ich złożenie jako pojedyncze wyrażenie lambda. Zademonstruj działanie składając trzy operacje: dodanie 1, podniesienie do kwadratu i pomnożenie przez 2.

30 Moduł `functools`

1. Wykorzystując `functools.reduce`, oblicz sumę elementów listy liczb całkowitych `values`. Wartość początkową akumulatora ustaw na 0.
2. Korzystając z `functools.reduce` oraz modułu `operator`, oblicz iloczyn elementów listy `numbers` zawierającej liczby zmiennoprzecinkowe. Nie używaj wyrażenia lambda.
3. Napisz funkcję `clip_value`, która przyjmuje wartość, dolny i górny limit oraz zwraca tę wartość przyciętą do podanego przedziału. Następnie przy pomocy `functools.partial` utwórz funkcję `clip_percent`, która ma limity ustawione na 0 i 100.
4. Zdefiniuj funkcję `format_number(value, precision, suffix)` zwracającą sformatowany ciąg znaków. Przy użyciu `functools.partial` skonfiguruj wariant `format_percent` z precyzją 2 i sufiksem `"%"` oraz wariant `format_currency` z precyzją 2 i sufiksem `" PLN"`.
5. Wykorzystując dekorator `functools.lru_cache` zaimplementuj rekurencyjnie funkcję `fibonacci(n)` zwracającą n-ty wyraz ciągu Fibonacciego. Wywołaj funkcję dla wartości 100 i wypisz wynik.
6. Zaimplementuj rekurencyjną funkcję `binomial(n, k)` obliczającą współczynnik dwumianowy. Użyj dekoratora `functools.cache`, a następnie po kilku wywołaniach wypisz statystyki z `cache_info`.
7. Korzystając z dekoratora `functools.lru_cache(maxsize=32)`, zaimplementuj funkcję `expensive_lookup(key)`, która symuluje kosztowne pobranie danych przez wywołanie `time.sleep(1)` i zwrócenie słownika. Zmierz czas drugiego wywołania z tym samym argumentem i porównaj z pierwszym.
8. Wyjaśnij w komentarzu, dlaczego funkcja z dekoratorem `lru_cache` nie zadziała, jeśli przekażemy do niej argument typu `list`. Następnie napisz funkcję `sum_unique(items)`, która przyjmuje krotkę i zwraca sumę unikalnych wartości, oraz pokaż użycie cache z różnymi krotkami jako wejściem.
9. Zdefiniuj klasę `Statistics` z konstruktorem przyjmującym sekwencję liczb. Użyj `functools.cached_property` do zdefiniowania atrybutów `mean`, `variance` i `stdev`. Wewnątrz każdej metody dodaj `print` informujący o uruchomieniu obliczeń, aby zweryfikować, że są wykonywane tylko raz.

10. Zaimplementuj klasę `TextAnalyzer` z konstruktorem przyjmującym ciąg znaków. Wykorzystując `functools.cached_property`, dodaj atrybuty `word_count`, `unique_words` (zwracający `set`) oraz `most_common` (zwracający najczęściej występujące słowo).

11. Korzystając z `functools.singledispatch`, zdefiniuj funkcję `describe(value)`, która dla argumentu typu `int` zwraca napis `"integer: ..."`, dla `float` napis `"float: ..."`, a dla `str` napis `"string of length ..."`. Dodaj implementację domyślną zgłaszającą `TypeError`.

12. Wykorzystując `functools.singledispatch`, napisz funkcję `serialize(data)`, która dla listy zwraca jej reprezentację jako ciąg znaków z elementami oddzielonymi przecinkami, dla słownika - parami `klucz=wartość` oddzielonymi średnikami, a dla krotki - elementami w nawiasach okrągłych.

13. Napisz dekorator `log_calls`, który wypisuje nazwę dekorowanej funkcji oraz przekazane argumenty przed każdym wywołaniem. Wykorzystaj `functools.wraps`, aby zachować metadane oryginalnej funkcji. Sprawdź, że `__name__` i `__doc__` udekorowanej funkcji są zachowane.

14. Zaimplementuj dekorator `retry(times)`, który ponawia wywołanie funkcji do `times` razy w przypadku zgłoszenia wyjątku. Wykorzystaj `functools.wraps`. Przetestuj dekorator na funkcji, która losowo zgłasza wyjątek lub zwraca wynik.

15. Korzystając z dekoratora `functools.total_ordering`, zaimplementuj klasę `Temperature` przechowującą wartość w stopniach Celsjusza. Zdefiniuj tylko metody `__eq__` i `__lt__`, a następnie sprawdź działanie wszystkich operatorów porównania oraz funkcji `sorted` na liście instancji.

16. Wykorzystując `functools.total_ordering`, zaimplementuj klasę `Version`, której instancje porównywane są na podstawie krotki `(major, minor, patch)`. Pokaż użycie operatorów `<`, `>=`, `==` oraz sortowania listy obiektów.

17. Wykorzystując `functools.reduce` oraz funkcję `operator.and_` z modułu `operator`, oblicz część wspólną listy zbiorów `sets_list`. Następnie zrób to samo dla sumy zbiorów, używając `operator.or_`.

18. Mając listę słowników `records`, w której każdy element zawiera klucze `category` i `amount`, wykorzystaj `functools.reduce`, aby zbudować słownik agregujący sumy kwot per kategoria. Akumulatorem ma być pusty słownik.

19. Napisz dekorator `memoize_with_stats`, który łączy memoizację z licznikiem trafień i nietrafień (bez użycia `lru_cache`). Wykorzystaj `functools.wraps`. Dekorator ma dodawać do funkcji metody `cache_hits` i `cache_misses` zwracające aktualne liczniki.

20. Korzystając z `functools.partial`, `functools.reduce` oraz `functools.singledispatch`, zaprojektuj funkcję `pipeline(value, *steps)` aplikującą sekwencję transformacji do wartości wejściowej. Następnie zdefiniuj `singledispatch`-ową funkcję `apply_default`

dającą różne kroki transformacji dla typów `int`, `str` i `list`, a `partial` wykorzystaj do skonfigurowania powtarzalnych wariantów kroków pipeline'u.